

# Batch Decryption without Epochs and its Application to Encrypted Mempools

Dan Boneh, Evan Laufer, and Ertem Nusret Tas

Stanford University

**Abstract.** Suppose Alice holds a secret key  $sk$  in a public key encryption scheme. For a given a set of ciphertexts, Alice wants to create a short pre-decryption key that lets anyone decrypt this exact set of ciphertexts and nothing else. This problem is called *batch decryption*. When the secret key  $sk$  is shared among a number of decryption parties the problem is called *batch threshold decryption*. This question comes up in the context of an encrypted mempool where the goal is to publish a short pre-decryption key that can be used to decrypt all ciphertexts in a block. Prior work constructed batch threshold decryption with some limitations.

In this work we construct three new batch decryption and batch threshold decryption schemes. We first observe that a key-policy ABE (KP-ABE) scheme directly gives a batch decryption scheme. However, the best KP-ABE schemes, which happen to be lattice-based, lead to relatively long pre-decryption keys. We then use very different techniques to construct a new lattice-based batch decryption scheme with substantially shorter pre-decryption keys. Our construction employs a recent lattice-based vector commitment due to Waters, Wee, and Wu. Finally, for completeness, we show that a trilinear map leads to a highly efficient threshold batch decryption scheme.

# Table of Contents

|          |                                                               |    |
|----------|---------------------------------------------------------------|----|
| <b>1</b> | <b>Introduction</b>                                           | 3  |
| 1.1      | An Application to Encrypted Mempools                          | 5  |
| 1.2      | Additional related Work                                       | 6  |
| <b>2</b> | <b>Technical Overview</b>                                     | 6  |
| 2.1      | Warm-up: Batch Decryption from Attribute-Based Encryption     | 6  |
| 2.2      | Our Batch Decryption System                                   | 6  |
| 2.3      | Thresholdizing the Pre-decryption Key                         | 10 |
| <b>3</b> | <b>Preliminaries</b>                                          | 10 |
| 3.1      | Batch Decryption                                              | 10 |
| 3.2      | Background on Lattices                                        | 12 |
| 3.3      | Shifted Multi-Preimage Trapdoor Sampler                       | 15 |
| <b>4</b> | <b>Our Batch Decryption System</b>                            | 17 |
| 4.1      | Correctness and Parameter Sizes                               | 20 |
| 4.2      | Security                                                      | 21 |
| 4.3      | Multi-bit Encryption and Adaptive Security                    | 27 |
| 4.4      | Reducing the Size of Public Key                               | 27 |
| <b>A</b> | <b>Extended Preliminaries</b>                                 | 30 |
| A.1      | Batch Threshold Decryption                                    | 30 |
| A.2      | BEAT-MEV: Batch (Threshold) Decryption by Bormet et al. [18]. | 31 |
| A.3      | Vector Commitment Scheme by Waters et al. [46]                | 33 |
| A.4      | Identity Based Encryption (IBE) Scheme by Gentry et al. [31]  | 33 |
| <b>B</b> | <b>Batch Decryption from Attribute Based Encryption</b>       | 35 |
| B.1      | Definitions                                                   | 35 |
| B.2      | The Construction                                              | 36 |
| B.3      | Security                                                      | 37 |
| B.4      | Parameter Sizes                                               | 37 |
| <b>C</b> | <b>Batch Decryption from Trilinear Maps</b>                   | 39 |
| C.1      | Background on Trilinear Maps and Generic Group Model          | 39 |
| C.2      | The Construction                                              | 39 |
| C.3      | Thresholdizing the Secret Key                                 | 40 |
| C.4      | Security                                                      | 40 |

# 1 Introduction

A public key encryption scheme that supports batch decryption is a tuple of four PPT algorithms ( $\text{SETUP}, \text{ENC}, \text{PREDEC}, \text{DEC}$ ) where

- $\text{SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{pk}, \text{sk})$ : takes as input a maximum batch size  $\ell$  and outputs a key pair  $\text{pk}, \text{sk}$ ;
- $\text{ENC}(\text{pk}, r, m) \rightarrow \text{ct}$ : encrypts  $m$  with respect to a unique identity tag  $r$ ;
- $\text{PREDEC}(\text{sk}, \{r_i\}_{i \in B}) \rightarrow \text{sbk}$ : takes as input a set  $B$  of up to  $\ell$  identity tags and outputs a corresponding short pre-decryption key  $\text{sbk}$ ; and
- $\text{DEC}(\text{pk}, \text{sbk}, \{\text{ct}_i\}_{i \in B}) \rightarrow \{m_i\}_{i \in B}$ : decrypts a batch of ciphertexts  $\{\text{ct}_i\}_{i \in B}$  using  $\text{sbk}$ .

In its simplest form, the encryptor can choose the identity tag  $r$  from a sufficiently large space to ensure that  $r$  is unique, with high probability, when all encryptors choose a random  $r$ . The key property of batch decryption is that the pre-decryption key  $\text{sbk}$  is short; its length should be at most polylog in the batch size  $|B|$ , and preferably independent of the batch size. This short pre-decryption key enables the decryption of all the ciphertexts  $\{\text{ct}_i\}_{i \in B}$  and nothing else. We give a precise definition in Section 3.1. Batch decryption is useful whenever many ciphertexts need to be decrypted concurrently by a third party. This most notably comes up in the context of an encrypted mempool, as explained in Section 1.1 below.

Batch decryption was introduced by Choudhuri, Garg, Piet, and Policharla [22]. In their scheme, however, each message was encrypted with respect to an identity tag  $r$  and an *epoch id*  $e$ , and only a *single* pre-decryption key can be published for every epoch  $e$ . Publishing more than one key per epoch, say for two sets  $B_1$  and  $B_2$ , would compromise security. This is called epoch-based batch decryption, and as we explain below, it makes the scheme quite difficult to use in the context of an encrypted mempool. The original scheme required a new setup at the beginning of every epoch, but this was later improved [44, 1, 23] to a single one-time setup at the beginning of time (but still only one pre-decryption key is allowed per epoch).

The first batch decryption scheme without epochs, called BEAT-MEV, was proposed by Bormet, Faust, Othman, and Qu [18]. BEAT-MEV supports the syntax presented above, where every message is encrypted with respect to an identity tag  $r$ , without the need for epochs. Pre-decryption keys can be published for an unlimited number of sets  $B$  of identity tags. The difficulty with BEAT-MEV is that it has a large public key that scales at least quadratically with the size of the identity space. One can shrink the public key by restricting the identity space to the maximum batch size  $\ell$ , however, this would require encryptors to coordinate in advance to select a unique identity tag for their messages. BEAT-MEV removes the coordination requirement by splitting transactions into multiple small batches of unique identities, and setting the pre-decryption key to be a vector of keys, one key per batch. This increases the size of the pre-decryption key, and more importantly, the public key remains at least quadratic in the batch size.

**Our contributions.** Our main contribution is an epoch-less batch decryption scheme with a succinct public key, ciphertexts, and pre-decryption keys.

In Section 4, we present our batch decryption scheme based on the Learning with Errors (LWE) assumption. At a high level, the construction follows the structure of prior epoch-based batch decryption schemes, which derive IBE secret keys for a vector commitment of a batch of identities. In particular, our scheme combines the IBE scheme of GPV [31, §7.2] with the linearly homomorphic vector commitment scheme of Waters, Wee, and Wu [46, §6]. As in the lattice IBE of [2], the lattices in our construction have two parts: a trapdoor for the left lattice is used to generate opening proofs

(and the secret key) for the vector commitment, while a trapdoor for the right lattice enables simulation of private keys for all batches excluding the adversary’s challenge identity.

In Appendix B, we observe that a batch decryption scheme can be built generically using key-policy attribute-based encryption (KP-ABE), which can be constructed from LWE and its variants [14,47]. Although both our construction in Section 4 and the KP-ABE construction in Appendix B achieve public key, ciphertext, and pre-decryption key sizes that are poly-log in the batch size, our scheme has pre-decryption keys that are *square root* the size of the pre-decryption keys in the construction based on KP-ABE (see Table 1).

**Batch Decryption from a Trilinear Map.** In Appendix C, we present a very efficient batch decryption system constructed using a trilinear map. It is based on the aggregate IBE scheme introduced by Goyal et al. [32, §7], and uses a trilinear map to shrink the linear-sized (in batch size) ciphertexts of the original scheme based on pairings. It achieves optimal parameter sizes: a single group element for the secret key, and an overhead of a single group element for the ciphertext.

**Comparison.** In Table 1, we present the comparison of the parameter sizes of four batch decryption systems. We denote the logarithm of the size of the identity space by  $k$ , and compare the parameters for the same value of  $k$  across different schemes. We observe that our batch decryption scheme from LWE has an exponential improvement over BEAT-MEV in terms of public key size. Asymptotically, both schemes have similar secret key and ciphertext sizes; however, BEAT-MEV has smaller pre-decryption keys and ciphertexts in concrete terms. Our batch decryption scheme also has a square-root improvement over the secret key size of the batch decryption scheme based on the BGGHNSVV14 KP-ABE [14]. It achieves smaller public key and ciphertext sizes compared to the batch decryption schemes based on the CW23 and Wee24 KP-ABEs [24,14], due to the

|                                      | pk                                               | sbk                         | ct                                               |
|--------------------------------------|--------------------------------------------------|-----------------------------|--------------------------------------------------|
| KP-ABE by BGGHNSVV14 [14]            | $O_\lambda(k \log^2(k))$                         | $O_\lambda(k \log^2(k))$    | $O_\lambda(k \log^2(k))$                         |
| KP-ABE by CW23 [24]                  | $O_\lambda(k \log^2(k))$                         | $O_\lambda(1)$              | $O_\lambda(k \log^2(k))$                         |
| KP-ABE by Wee24, $\alpha = 1/3$ [47] | $k^{2/3} \text{poly}(\lambda, \log(k))$          | $\text{poly}(\lambda)$      | $k^{2/3} \text{poly}(\lambda, \log(k))$          |
| KP-ABE by Wee24, $\alpha = 1$ [47]   | $k^2 \text{poly}(\lambda, \log(k))$              | $\text{poly}(\lambda)$      | $\text{poly}(\lambda, \log(k))$                  |
| BEAT-MEV – SB [18, §5.3]             | $O(\ell^2  \mathbb{G}_1  + \ell  \mathbb{G}_2 )$ | $\log(\ell)  \mathbb{G}_2 $ | $\log(\ell) (3  \mathbb{G}_1  +  \mathbb{G}_T )$ |
| BEAT-MEV – SB*                       | $O(\ell^2  \mathbb{G}_1  + \ell  \mathbb{G}_2 )$ | $ \mathbb{G}_2 $            | $\log(\ell) (3  \mathbb{G}_1  +  \mathbb{G}_T )$ |
| BEAT-MEV – BLMR (Sec. A.2)           | $O_\lambda(1)$                                   | $O_\lambda(k^2)$            | $O_\lambda(k^3)$                                 |
| Our BD Scheme from LWE – SB          | $O_\lambda(\log^2(\ell))$                        | $O_\lambda(\log^2(\ell))$   | $O_\lambda(\log^4(\ell))$                        |
| Our BD Scheme from LWE – SB*         | $O_\lambda(\log^2(\ell))$                        | $O_\lambda(\log(\ell))$     | $O_\lambda(\log^4(\ell))$                        |
| Our BD Scheme from Trilinear Map     | $ \mathbb{G}'_1  + (\ell - 1)  \mathbb{G}'_2 $   | $ \mathbb{G}'_3 $           | $ \mathbb{G}'_1  + O_\lambda(1)$                 |

Table 1: Comparison of the parameter sizes of four batch decryption systems without epochs. |pk| denotes the size of the public key, |sbk| denotes the size of the pre-decryption key published per batch, and |ct| denotes the ciphertext size. We denote the security parameter by  $\lambda$  and the maximum batch size by  $\ell$ . We denote the sizes of the  $\mathbb{G}_1$ ,  $\mathbb{G}_2$  and  $\mathbb{G}_T$  group elements used by BEAT-MEV [18] by  $|\mathbb{G}_i|$ ,  $i = 1, 2, T$  and the sizes of the  $\mathbb{G}_1$ ,  $\mathbb{G}_2$ ,  $\mathbb{G}_3$  and  $\mathbb{G}_T$  group elements used by the trilinear map by  $|\mathbb{G}'_i|$ , for  $i = 1, 2, 3, T$ . Finally,  $k$  denotes the logarithm of the size of the identity space. To simplify the expressions, we assume that  $\ell = \text{poly}(\lambda)$  and  $\ell \geq \lambda$ . Also note that  $k > \log(\ell)$  for the KP-ABE schemes. In the table, SB stands for sub-batches and refers to setting  $k = \log_2(\ell)$  and applying the technique in [18, §5.3] to resolve the issue of collision at the same identity tags due to the small size of the identity space. SB\* refers to the method explained in Section 4.4 that overcomes the  $\log_2(\ell)$  factor increase in the ciphertext size due to [18, §5.3]. The  $\alpha$  parameter of the KP-ABE by Wee24 [47] enables trading-off the public key and ciphertext sizes. All of the ciphertext sizes except BEAT-MEV-SB have an additive  $k$  term that denotes the size of the selected identity.

latter’s dependence on  $k$  (see Appendix B.4). We note that the KP-ABEs [14,24,47] do not use the random oracle model.

*Remark 1 (lattice-based BEAT-MEV).* The large public key size of BEAT-MEV is due to the use of a specific pairing-based *key-homomorphic* PRF that is also puncturable (see Appendix A.2 for a description of BEAT-MEV). We observe that one can achieve a smaller public key using alternative puncturable key-homomorphic PRF constructions, such as that of Boneh, Lewi, Montgomery, and Raghunathan (BLMR) [16, §5]. Using BLMR, the base BEAT-MEV scheme would have a public key of size  $O_\lambda(1)$ , instead of  $2^k$  as in BEAT-MEV, but a ciphertext of size  $O_\lambda(k^3)$  and a pre-decryption key of size  $O_\lambda(k^2)$ , which are worse than BEAT-MEV (see Appendix A.2).

## 1.1 An Application to Encrypted Mempools

When a transaction is posted to a blockchain, it first goes into a public mempool and waits there until it is eventually included in a block that is committed on chain. While the transaction is in the mempool, it is visible to the world and this can lead to front-running attacks that cause financial loss for the transaction issuer. This issue is called MEV [26]. An encrypted mempool [39,5] is a way to protect transactions while they are in the mempool. Instead of submitting transactions in the clear, the transaction issuer encrypts the transaction with a public key  $\text{pk}$  and submits it to the mempool. The corresponding secret key  $\text{sk}$  is shared among the nodes that operate the blockchain. Whenever a block is committed on chain, the nodes publish decryption shares that enable anyone to decrypt and execute all the transactions in that block. This way, a transaction becomes public only after it is included in a block that is committed on chain. Encrypted mempools are already in use in some blockchains such as Osmosis and Shutter [30].

Batch decryption is a way to greatly improve the efficiency of an encrypted mempool. With a standard public key encryption, for every block, a blockchain node would need to publish  $\ell$  decryption shares to enable the decryption of all  $\ell$  transactions in a block. In other words, for every block the node must publish  $O(\lambda\ell)$  bits to enable decryption of that block. With batch decryption, or more precisely batch threshold decryption, the node would publish a short pre-decryption key whose length is only  $O(\lambda \text{polylog}(\ell))$  or just  $O(\lambda)$ . The prior batch decryption schemes [23,22,1,44,18] were proposed for exactly this purpose. In epoch-based batch decryption, the block number (a.k.a block height) would be used as the epoch. To use such a scheme, the transaction issuer would need to guess the block number where its transaction would land, and encrypt using that epoch (and random identity tag). If the issuer guesses wrong, then the transaction would never be included in a block, and must be resubmitted. Epoch-free schemes eliminate this difficulty.

When describing our construction in Section 4, we focus on (non-threshold) batch decryption. In Appendix A, we review the formal definition of epoch-free batch threshold decryption. In Section 2.3, we explain how to thresholdize the keys in our batch decryption schemes.

**The Risk of Replicated Identities.** Since batch decryption schemes identify a ciphertext by its identity tag  $r$ , an encrypted mempool is vulnerable to an attacker who replicates the identity tag of an honest user’s transaction. Concretely, suppose an adversary observes in the mempool a ciphertext generated by an honest user under identity tag  $r$ . The adversary can create a *dummy* ciphertext encrypted under the *same* identity  $r$ . If this dummy ciphertext is included in a block, the pre-decryption key generated for that block can be used to decrypt the honest user’s ciphertext before it is committed on chain. Solutions to this problem were discussed in the earlier works [23,22,1,44,18].

For example, the transaction issuer can generate a key pair for a signature scheme, use the public key as the identity tag, and post the encrypted transaction along with a signature on the encrypted transaction. The nodes will only include a transaction in a block if it has a valid signature with respect to its identity tag. This way, the adversary cannot post a dummy transaction with the same identity tag.

Of course, to prevent malleability attacks of a similar nature, it is crucial that batch decryption schemes provide CCA2 security. To this end, standard techniques (e.g., [12,10,11]) can be employed to elevate threshold CPA security of batch decryption to full threshold CCA2 security.

## 1.2 Additional related Work

Early work on batch decryption [36,28] proposed to use Identity Based Encryption (IBE), where a transaction issuer encrypts to an *epoch id* as the IBE identity. Blockchain nodes then release the corresponding IBE secret key once the block at that epoch id is committed on chain. These mechanisms, however, lack the ability to selectively exclude ciphertexts from the batch, as well as any security guarantees for such excluded ciphertexts. Consequently, they are not suitable for encrypted mempools.

## 2 Technical Overview

### 2.1 Warm-up: Batch Decryption from Attribute-Based Encryption

Key-policy attribute-based encryption (KP-ABE) [40,33] generalizes the public-key encryption paradigm by associating each secret key  $\text{sk}_f$  with a function  $f: \mathcal{X} \rightarrow \mathcal{Y}$ . Ciphertexts are labeled with a public attribute  $x \in \mathcal{X}$  and can be decrypted using  $\text{sk}_f$  if and only if  $f(x) = 0$ .

Batch decryption can be achieved by any KP-ABE scheme instantiated with a special class of functions. These functions  $f_S$  take an identity as input and return 0 if the identity belongs to a designated set of identities  $S = \{r_i\}_{i \in [\ell]}$  associated with the key. Hence, a ciphertext labeled with an identity  $r$  as its public attribute can be decrypted by the key  $\text{sk}_{f_S}$  if and only if  $r \in S$ , *i.e.*, if  $f_S(r) = 0$ . This batch decryption system is formally presented in Appendix B, where its security is shown to reduce to the security of the underlying KP-ABE scheme. For comparison of parameter sizes with our BD scheme, we consider the KP-ABE construction by Boneh et al. [14], which has succinct secret keys and supports arithmetic circuits with arbitrary fan-in gates.

### 2.2 Our Batch Decryption System

**2.2.1 Batch Decryption as Batch Identity Based Encryption.** Although KP-ABE directly implies batch decryption, it does so at the expense of large pre-decryption keys (*cf.* Table 1). An alternative approach to building practical batch decryption schemes (with epochs) has been extending efficient identity-based encryption (IBE) schemes to the batch setting [1,23]. IBE is a public-key encryption system, where public keys can be arbitrary strings [42,13]. Practical IBE schemes have been realized under various cryptographic assumptions, including the bilinear Diffie-Hellman problem [13], quadratic residuosity modulo composites [25,15], and LWE [31,2]. In the IBE framework, decryption of a ciphertext associated with an identity  $r$  requires the recipient to obtain the corresponding secret key  $\text{sk}_r$  from a trusted generator, which derives  $\text{sk}_r$  using a master secret key  $\text{msk}$ .

Unlike traditional IBE schemes, where each identity is associated with a distinct secret key, batch decryption schemes seek to obtain a single secret key, called the pre-decryption key, capable of decrypting ciphertexts associated with multiple identities (thus motivating the term batch IBE [1]). The main requirement in this setting is to ensure that both the pre-decryption key and ciphertexts remain succinct in the size of the identity batch. To achieve succinct pre-decryption keys, earlier batch decryption systems (with epochs) [1,23] exploit the observation that IBE implies a signature scheme [13, §6]. Specifically, they generate pre-decryption keys in the form of a signature on an accumulator value committing to a batch of identities [1,23]. Then, the batch decryption functionality is realized using witness encryption for two verification relations: (i) the signature verification relation induced by the underlying IBE scheme, and (ii) a membership verification relation asserting that a given identity belongs to the accumulator. Here, the public inputs are the identities  $r$ , while the witness comprises the accumulator value  $d$ , a signature  $\sigma$  over  $d$ , and membership proofs  $\pi_r$  for each  $r$  with respect to  $d$ .

Concrete instantiations of this paradigm by Agarwal et al. [1], Choudhuri et al. [23] and Suegami et al. [44] employ BLS signatures for signing and KZG commitments to implement the accumulator. For witness encryption, their schemes rely on public linear constraint systems in which group elements appearing in the witness are never paired with one another. Enforcing this linearity condition requires the use of a ‘one-time BLS signature’ (without random oracles) tied to a specific ‘epoch’, thereby necessitating encryption with respect to both identities and a pre-determined epoch number – an essential aspect for preserving security in their constructions.

In our batch decryption scheme (without epochs), we follow the same blueprint as the previous schemes and generate IBE secret keys for vector commitments to a batch of identities. It can thus be viewed as a batch IBE scheme based on the IBE protocol by Gentry et al. [31, §7.2] and the linearly-homomorphic vector commitment scheme by Waters et al. [46, §6] (*cf.* Appendices A.3 and A.4 for a summary of these protocols). We overcome the limitation of epochs, as the signature scheme inherent to our choice of IBE, *i.e.*, GPV signatures, supports verification using only linear operations, in addition to checking the signature norm. We next briefly look at the main components of our construction and the steps taken towards the final scheme.

**2.2.2 The Vector Commitment Scheme of Waters, Wee, and Wu [46].** Vector commitment schemes enable committing to a sequence of messages such that the correctness of any message at a given index can be efficiently verified via an opening proof [21,34,20]. In our BD construction, we use the lattice-based vector commitment scheme by Waters et al. [46, §6] (*cf.* Appendix A.3). The scheme follows the general framework for constructing lattice-based vector commitments from earlier works [37,3,48]. While earlier works could only commit to vectors with small entries [37,3] or required public parameters quadratic in the size of the vector [48], Waters et al. overcome these limitations by constructing a *shifted multi-preimage trapdoor sampler* with respect to a structured collection of matrices  $\mathbf{A}_1, \dots, \mathbf{A}_\ell$ . The matrices  $\mathbf{A}_i$  are correlated but have marginally uniform distributions  $\mathbb{Z}_q^{n \times (m \lceil \log \ell \rceil)}$ . Concretely, for each  $i \in [\ell]$ ,  $\mathbf{A}_i := [\mathbf{A}_0 \mid \mathbf{B} - u_i \otimes \mathbf{G}]$  for uniformly random  $\mathbf{A}_0, \mathbf{B}$  and binary-encoded  $u_i$ . The sampler does not need any auxiliary hints and can obtain a gadget trapdoor for the matrix

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 & & & -\mathbf{G} \\ & \mathbf{A}_2 & & -\mathbf{G} \\ & & \dots & \dots \\ & & & \mathbf{A}_\ell - \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{n\ell \times (\ell m \lceil \log \ell \rceil + m)}$$

without any hints for  $\mathbf{A}_i$ ,  $i \in [\ell]$ .

The public parameters consist of the matrices  $\mathbf{A}_0$  and  $\mathbf{B}$ . Given these matrices and the values  $(x_1, \dots, x_\ell)$  to be committed, the commitment algorithm constructs the matrix  $\mathbf{A}$ , sets  $u_i$  to be binary representations of  $i$ , and samples a shifted preimage for the column of vectors  $(x_1 \mathbf{e}_1, \dots, x_\ell \mathbf{e}_\ell)$  using trapdoor  $\mathbf{td}$  for  $\mathbf{A}$ :

$$\text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\mathbf{td}, x_1 \mathbf{e}_1, \dots, x_\ell \mathbf{e}_\ell) \in \mathbb{Z}_q^{m\ell \lceil \log \ell \rceil + n}$$

Here,  $\mathbf{e}_1 = [1, 0, \dots, 0]^\top \in \mathbb{Z}_q^m$ ,  $\mathbf{v}_i$ ,  $i \in [\ell]$ , are the opening proofs for  $x_i$ , and  $\mathbf{c}$  is the commitment. Opening proofs are verified by checking that  $\mathbf{v}_i$  are low-norm vectors and  $\mathbf{A}_i \mathbf{v}_i = \mathbf{c} - x_i \mathbf{e}_1$ , where the pre-images must be shifted by  $\mathbf{c}$ .

**2.2.3 The GPV IBE System [31].** The IBE uses a random oracle  $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$  to associate identities with public keys of the dual-Regev cryptosystem.

The setup algorithm generates a matrix-trapdoor tuple  $(\mathbf{C}, \mathbf{T}_\mathbf{C})$  and sets the master public key to be  $\mathbf{C}$  and the master secret key to be  $(\mathbf{C}, \mathbf{T}_\mathbf{C})$ . Then, to generate the secret key for an identity  $r$ , it computes  $\mathbf{h}_r = H(r)$  and returns the corresponding decryption key  $\mathbf{sk}_r \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_\mathbf{C}, \mathbf{h}_r, \sigma)$ . Here,  $\text{SamplePre}$  outputs a discrete Gaussian sample  $\mathbf{v}$  with width  $\sigma$  that satisfies  $\mathbf{C} \cdot \mathbf{v} = \mathbf{h}_r$ .

To encrypt a bit  $m \in \{0, 1\}$  for identity  $r$ , the encryption algorithm generates a dual-Regev ciphertext  $\text{ct} = (\text{ct}[1], \text{ct}[2])$  with  $\mathbf{h}_r = H(r)$  as the public key and  $\mathbf{C}$  as the public parameter:

$$\begin{aligned} \text{ct}[1] &:= \mathbf{h}_r^\top \mathbf{s} + \epsilon_2 + m \lfloor q/2 \rfloor \in \mathbb{Z}_q^m \\ \text{ct}[2] &:= \mathbf{C}^\top \mathbf{s} + \epsilon_1 \in \mathbb{Z}_q, \end{aligned}$$

where  $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ , and  $\epsilon_1, \epsilon_2$  are low-norm noise vectors.

Finally, decryption outputs  $\text{Round}(\text{ct}[1] - \mathbf{sk}_r^\top \text{ct}[2])$ , where  $\text{Round}(x)$  outputs 0 if  $x \bmod q$  is closer to 0 than to  $\lfloor q/2 \rfloor$ . Correctness follows from the fact that  $\mathbf{C} \cdot \mathbf{sk}_r = \mathbf{h}_r$  and  $\mathbf{sk}_r$  is a low-norm vector.

**2.2.4 A First Attempt.** To generate the pre-decryption key for a batch of identities  $r_1, \dots, r_\ell$ , the batch decryption scheme first computes a vector commitment  $\mathbf{c}$  to the sequence  $(\mathbf{h}_1, \dots, \mathbf{h}_\ell)$  of random vectors, where  $\mathbf{h}_i = H(r_i) \in \mathbb{Z}_q^n$ ,  $i \in [\ell]$ :

$$\text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\mathbf{td}, \mathbf{h}_1, \dots, \mathbf{h}_\ell) \in \mathbb{Z}_q^{m\ell \lceil \log \ell \rceil + n},$$

It then returns as pre-decryption key  $\mathbf{sk}_c \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_\mathbf{C}, \mathbf{c}, \sigma)$ , where the matrix  $\mathbf{C} \in \mathbb{Z}_q^{n \times m}$  is part of the public parameters, and the corresponding trapdoor  $\mathbf{T}_\mathbf{C}$  is the master secret key in IBE terms.

Now, to encrypt a bit  $m \in \{0, 1\}$  to the identity  $r_i$  and index  $i$ , the encryption algorithm generates a dual-Regev ciphertext  $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \text{ct}_i[3])$  with  $\mathbf{h}_r = H(r)$  as the public key and  $(\mathbf{A}_i, \mathbf{C})$  as the public parameters:

$$\begin{aligned} \text{ct}_i[1] &:= \mathbf{h}_i^\top \mathbf{s} + \epsilon_1 + m \lfloor q/2 \rfloor \in \mathbb{Z}_q^m \\ \text{ct}_i[2] &:= \mathbf{C}^\top \mathbf{s} + \epsilon_2 \in \mathbb{Z}_q^m, \\ \text{ct}_i[3] &:= \mathbf{A}_i^\top \mathbf{s} + \epsilon_3 \in \mathbb{Z}_q^{m \lceil \log \ell \rceil}, \end{aligned}$$



where  $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ , and  $\epsilon_1, \epsilon_2, \epsilon_3$  are low-norm noise vectors.

Finally, the pre-decryption key to decrypt  $\text{ct}_i$  consists of the pre-decryption key  $\text{sk}_c$  for the batch and the opening proof  $\mathbf{v}_i$ . The decryptor calculates

$$\text{Round}(\text{ct}[1] - \text{sk}_c^\top \text{ct}[2] + \mathbf{v}_i^\top \text{ct}[3]),$$

which outputs the plaintext message since  $\mathbf{C} \cdot \text{sk}_c - \mathbf{A}_i \mathbf{v}_i = \mathbf{c} - \mathbf{A}_i \mathbf{v}_i = \mathbf{h}_i$ . To ensure that the decryptors obtain the same opening proofs  $\mathbf{v}_i$ , the scheme uses a hash function  $H_{\text{sp}}$  in lieu of a direct call to the `SampleMultPre` algorithm.

**2.2.5 Removing the Index Dependency.** The scheme described so far encrypts a message with respect to both an identity and an index. To remove this limitation, we select the binary vectors  $u_i$  used to construct  $\mathbf{A}_i$  as a function of the identities from a larger space  $\beta[\log(\ell)]$ , where  $\beta$  is an integer greater than 1. Then, given the  $\text{crs} = [\mathbf{A}_0 \mid \mathbf{B}]$  and the identity  $r$ , the encryptor sets its binary vector as  $\tilde{h}_r$ , where  $(\mathbf{h}_r, \tilde{h}_r) := H(r)$ , and the matrix  $\mathbf{A}_r$  used in encryption as  $\mathbf{A}_r := [\mathbf{A} \mid \mathbf{B} - \tilde{h}_r \otimes \mathbf{G}]$ . When finding the pre-decryption key corresponding to a sequence of identities  $\{r_i\}_{i \in [\ell]}$ , the matrix  $\mathbf{A}$  can be composed from  $\mathbf{A}_{r_i}$ ,  $i \in [\ell]$ , in any pre-determined order.

The parameter  $\beta$  comes with a trade-off: larger  $\beta$  implies larger public parameters and ciphertexts (linear in  $\beta$ ), but removes the need for the encryptors to de-duplicate their indices. In Section 4.4, we propose a practical solution that obviates any coordination requirement with a blow-up factor of only  $\log(\ell)$ .

**2.2.6 The Final Attempt: Simulating Pre-decryption Key Queries.** In the selective IBE security game, the adversary specifies a challenge identity  $r^*$  (with hash  $\mathbf{h}^*$  and  $\tilde{h}^*$ ) at the outset of the game. The challenger is then responsible for simulating the random oracle and answering pre-decryption key queries for batches of identities, with the constraint that  $r^*$  is excluded from all such batches. Importantly, the challenger must perform these simulations without knowledge of the pre-decryption keys corresponding to any batch containing  $r^*$ , and without knowledge of the trapdoors for the matrices  $\mathbf{A}_{r^*}$  and  $\mathbf{C}$ .

Note that the challenger can respond to pre-decryption key queries without a trapdoor for  $\mathbf{C}$  by sampling a random, low-norm key  $\text{sk}_c$  and running the key generation algorithm of the dual-Regev scheme with the public parameter  $\mathbf{C}$  and public key  $\mathbf{c} := \mathbf{C} \cdot \text{sk}_c$ . However, in this case, it must keep its response to  $H_{\text{sp}}$  queries consistent with the commitment  $\mathbf{c}$  generated before. To ensure this, we use the key sampling technique from [2, §6]. Given an input  $\mathbf{h} = [\mathbf{h}_1, \dots, \mathbf{h}_\ell]^\top$ , we extend each matrix  $\mathbf{A}_{r_i}$  within  $\mathbf{A}$  as  $[\mathbf{A}_{r_i} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_i) \tilde{\mathbf{B}}]$ , where  $H_{\text{frd}}(\cdot)$  is an encoding with full-rank differences ([2, §5]), and  $\tilde{\mathbf{A}}, \tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times m \lceil \log(\ell) \rceil}$  are randomly-sampled matrices:

$$[\mathbf{A} \mid \tilde{\mathbf{A}}] = \begin{bmatrix} \mathbf{A}_{r_1} & -\mathbf{G} \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_1) \tilde{\mathbf{B}} & & & \\ & \mathbf{A}_{r_2} & -\mathbf{G} & \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_1) \tilde{\mathbf{B}} & \\ & & \dots & & \\ & & & \dots & \\ & \mathbf{A}_{r_\ell} & -\mathbf{G} & & \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_\ell) \tilde{\mathbf{B}} \end{bmatrix}$$

In normal operation, the key generator uses the trapdoor  $\text{td}$  for  $\mathbf{A}$  and the algorithm `SampleLeft` from [2, §4] to find the shifted pre-images  $\mathbf{v}_i$  of the target vectors  $\mathbf{h}_i$  over  $[\mathbf{A} \mid \tilde{\mathbf{A}}]$ . In the simulation, the challenger sets  $\tilde{\mathbf{A}} := \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}$ , where  $\tilde{\mathbf{R}}^* \leftarrow \{1, -1\}^{m \times m}$ . Then, using a trapdoor  $\mathbf{T}_{\tilde{\mathbf{B}}}$  for  $\tilde{\mathbf{B}}$  (but not for  $\mathbf{A}_{r^*}$ ,  $\mathbf{A}$  or  $\mathbf{C}$ ), it can invoke the algorithm `SampleRight` from [2, §4] to find

$\mathbf{v}_i$ ,  $i \in [\ell]$ , as long as  $r_i \neq r^*$  for any  $i \in [\ell]$ . Here, by the generalized leftover hash lemma from [2, §3], the matrix  $\mathbf{A}_{r^*} \tilde{\mathbf{R}}^*$  is uniform and independent from other matrices, implying that the matrix

$$[\mathbf{A}_{r^*} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}] = [\mathbf{A}_{r^*} \mid \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*]$$

can be replaced by LWE samples for a security reduction.

### 2.3 Thresholdizing the Pre-decryption Key

To achieve batch threshold decryption, trapdoor for the matrix  $\mathbf{C}$  can be shared among a set of  $N$  parties, and pre-images of the vector commitments can be sampled collectively using the techniques in [7, §3]. When  $N$  is small, the batch decryption scheme can also be thresholdized without sacrificing performance by applying the combinatorial transformation in [17, §22.3.1]. This requires each encryptor to run a  $t$ -out-of- $N$  secret sharing on its message  $m$  and to encrypt each message share using the public key of a different party. This increases the ciphertext and public parameter sizes by a factor of  $N$ .

We present our batch decryption construction, its correctness and security proof, and estimate its parameter, ciphertext and pre-decryption key sizes in Section 4.

## 3 Preliminaries

**Notation.** We use  $\lambda$  as the security parameter. We call a function  $f(\cdot)$  negligible (in  $\lambda$ ), denoted by  $\text{negl}(\lambda)$ , if  $f(\lambda) = o(1/\text{poly}(\lambda))$  for every polynomial  $\text{poly}(\cdot)$ . We adopt the short-hands  $\mathcal{A}$  and  $\mathcal{B}$  for the adversary and Ch for the challenger. We denote computationally indistinguishable games by  $\approx_c$  and statistically indistinguishable games by  $\approx_s$ . We denote by  $[n]$  the set  $\{1, \dots, n\}$  and by  $\{x_{a_i}\}_{a_i \in \{a_1, \dots, a_k\}}$  the sequence  $(x_{a_1}, \dots, x_{a_k})$  such that  $a_1 < \dots < a_k$ . The notation  $a \leftarrow S$  implies a uniformly-random sample from the set  $S$ , unless the distribution over  $S$  is specified.

### 3.1 Batch Decryption

**Definition 1.** A **batch decryption** (BD) scheme instantiated with a maximum batch size of  $\ell \in \mathbb{N}$ , a message space  $\mathcal{M}$  and an identity space  $\mathcal{I}$  consists of four PPT algorithms:

- $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\mathbf{pk}, \mathbf{sk})$ : The setup algorithm takes as input the security parameter  $\lambda$  and the batch size  $\ell$ , and outputs a public key  $\mathbf{pk}$  and a secret key  $\mathbf{sk}$ .
- $\text{BD.ENC}(\mathbf{pk}, r, m) \rightarrow \mathbf{ct}$ : The encryption algorithm takes as input a public key  $\mathbf{pk}$ , an identity  $r$  and a message  $m \in \mathcal{M}$ , and outputs a ciphertext  $\mathbf{ct}$ .
- $\text{BD.PREDEC}(\mathbf{sk}, \{r_i\}_{i \in [\ell]}) \rightarrow \mathbf{sbk}$ : The predecryption algorithm takes as input the secret key  $\mathbf{sk}$  and  $\ell$  identities, and outputs a predecryption key  $\mathbf{sbk}$ .
- $\text{BD.DEC}(\mathbf{pk}, \mathbf{sbk}, \{\mathbf{ct}_i\}_{i \in [\ell]}) \rightarrow \{m_i\}_{i \in [\ell]}$ : The decrypt algorithm takes as input the predecryption key  $\mathbf{sbk}$  and  $\ell$  ciphertext, and outputs the corresponding messages.

A BD scheme must satisfy correctness, security under a chosen plaintext attack (CPA-security) and efficiency, such that the size of the predecryption key  $\mathbf{sbk}$  is succinct in the batch size  $\ell$ .

**Definition 2 (Correctness of BD).** For all  $\lambda \in \mathbb{N}$ ,  $\ell \in \mathbb{N}$ ,  $\{m_i\}_{i \in [\ell]} \in \mathcal{M}^\ell$  and  $\{r_i\}_{i \in [\ell]} \in \mathcal{I}^\ell$ , it holds that

$$\Pr \left[ \text{BD.DEC}(\text{pk}, \text{sbk}, \{\text{ct}_i\}_{i \in [\ell]}) = \{m_i\}_{i \in [\ell]} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell) \\ \text{ct}_i \leftarrow \text{BD.ENC}(\text{pk}, r_i, m_i) \quad \forall i \in [\ell] \\ \text{sbk} \leftarrow \text{BD.PREDEC}(\text{sk}, \{r_i\}_{i \in [\ell]}) \end{array} \right] > 1 - \text{negl}(\lambda)$$

**Definition 3 (Security of BD).** The security game  $\text{Game}_{\mathcal{A},b}^{\text{BD}}(1^\lambda, \ell)$  is defined below with respect to the adversary  $\mathcal{A}$ . A BD scheme is selectively secure in the random oracle model, if for all  $\ell = \text{poly}(\lambda)$  and PPT adversaries  $\mathcal{A}$ , there exists a negligible function  $\text{negl}(\cdot)$  such that

$$|\Pr[\text{Game}_{\mathcal{A},0}^{\text{BD}}(1^\lambda, \ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD}}(1^\lambda, \ell) = 1]| < \text{negl}(\lambda)$$

### The Security Game $\text{Game}_{\mathcal{A},b}^{\text{BD}}(1^\lambda, \ell)$

**Challenge identity:** The adversary commits to the challenge identity  $r^*$  and sends it to the challenger.

**Setup:** The challenger Ch takes as input the security parameter  $\lambda$  and the maximum batch size  $\ell$ . It runs  $(\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell)$  and sends  $\text{pk}$  to  $\mathcal{A}$ . The rest of the game proceeds in rounds, as follows:

**Pre-challenge queries:** The adversary may issue an arbitrary number of pre-challenge queries for computing the predecryption key:

- $\mathcal{A}$  sends a list  $\{r_i\}_{i \in [\ell]}$  of  $\ell$  identities to Ch.
- Ch computes the predecryption key  $\text{sbk} \leftarrow \text{BD.PREDEC}(\text{sk}, \{r_i\}_{i \in [\ell]})$  and sends  $\text{sbk}$  to  $\mathcal{A}$ .

**Challenge round:** At any point in time,  $\mathcal{A}$  may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$  sends two messages  $m_0, m_1 \in \mathcal{M}$  on which, it would like to be challenged.
- If the adversary has previously asked for the predecryption key corresponding to a list of identities  $\{r_i\}_{i \in [\ell]}$  such that  $r^* \in \{r_i\}_{i \in [\ell]}$ , then Ch halts the game.
- Otherwise, Ch computes  $\text{ct}_b \leftarrow \text{BD.ENC}(\text{pk}, r^*, m_b)$  for  $b \leftarrow \{0, 1\}$  and sends  $\text{ct}_b$  to  $\mathcal{A}$ .

**Post-challenge queries:** The adversary can again send an arbitrary number of key computation queries as in the pre-challenge case.

**Random oracle queries:** Upon receiving a query containing a new input  $x$ , Ch samples a value  $y$  from the output distribution of the random oracle, stores the tuple  $(x, y)$ , and returns  $y$ . It returns the value  $y$  upon receiving the same input  $x$  in a future query.

**Output:** At any point in time,  $\mathcal{A}$  can decide to output a bit  $b' \in \{0, 1\}$ . The game then halts with the same output  $b'$ .

A fully secure BD scheme is defined in a similar manner, with the distinction that the adversary is allowed to select the challenge identity  $r^*$  after observing the public key  $\text{pk}$  and issuing a polynomial number of pre-challenge queries.

### 3.2 Background on Lattices

This section is based on lemmas from [31], [2] and [46]. We assume all vectors are column vectors. Given vectors  $\mathbf{v}_1, \dots, \mathbf{v}_\ell \in \mathbb{Z}^m$ , we define the following short-hand notation:

$$\text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell) = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \dots \\ \mathbf{v}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell m}$$

Similarly, given the matrices  $\mathbf{A}_1, \dots, \mathbf{A}_\ell \in \mathbb{Z}^{n \times m}$ , we define the short-hand

$$\text{diag}(\mathbf{A}_1, \dots, \mathbf{A}_\ell) = \begin{bmatrix} \mathbf{A}_1 & & \\ & \mathbf{A}_2 & \\ & & \dots \\ & & & \mathbf{A}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell n \times \ell m}$$

We use the  $\ell_2$ -norm for vectors and matrices unless stated otherwise, *i.e.*,  $\|\mathbf{v}\| = \|\mathbf{v}\|_2 = \sqrt{\sum_i |\mathbf{v}_i|^2}$  for a vector  $\mathbf{v} = \text{col}(\mathbf{v}_1, \dots, \mathbf{v}_n) \in \mathbb{Z}^n$ , and  $\|\mathbf{A}\| = \max_{i \in [m]} \|\mathbf{v}_i\|$  for a matrix  $\mathbf{A} = [\mathbf{v}_1, \dots, \mathbf{v}_m] \in \mathbb{Z}^{n \times m}$ , where  $\mathbf{v}_i \in \mathbb{Z}^n$ ,  $i \in [m]$ . We denote by  $\|\mathbf{A}\|_{\text{GS}}$  the  $\ell_2$ -norm of the Gram-Schmidt orthogonalization of the matrix  $\mathbf{A}$ , and by  $\mathbf{I}_n \in \mathbb{Z}_q^{n \times n}$  the  $n$ -by- $n$  identity matrix. Note that for any matrix  $\mathbf{T} \in \mathbb{Z}_q^{m \times m'}$ , it holds that  $\|\mathbf{T}\|_{\text{GS}} \leq \|\mathbf{T}\| \leq m \|\mathbf{T}\|_\infty$ .

For a full-rank matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , the  $n$ -dimensional lattice generated by  $\mathbf{A}$  is given by

$$\Lambda(\mathbf{A}) = \{\mathbf{A}\mathbf{v} : \mathbf{v} \in \mathbb{Z}^n\}.$$

When  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  for some integers  $n, m, q$ , we define the following  $q$ -ary lattices:

$$\begin{aligned} \Lambda^{\mathbf{u}}(\mathbf{A}) &= \{\mathbf{v} \in \mathbb{Z}_q^m : \mathbf{A}\mathbf{v} = \mathbf{u} \bmod q\} \\ \Lambda(\mathbf{A}) &= \{\mathbf{y} \in \mathbb{Z}_q^m : \mathbf{y} = \mathbf{A}^\top \mathbf{v} \bmod q, \mathbf{v} \in \mathbb{Z}_q^n\} \end{aligned}$$

We denote by  $\lambda_1^\infty(\mathbf{A}) := \min_{\mathbf{v} \neq 0} \|\mathbf{A}^\top \mathbf{v}\|_\infty$ , the  $\ell_\infty$  norm of the shortest non-zero vector in the lattice defined by the matrix  $\mathbf{A}$ .

**Lemma 1 (Lemma 15 of [2]).** *Let  $\mathbf{R} \leftarrow \{-1, 1\}^{k \times m}$ . Then, there exists a universal constant  $C$  such that*

$$\Pr \left[ \|\mathbf{R}\| > C\sqrt{k+m} \right] < e^{-(k+m)}.$$

**3.2.1 Discrete Gaussians Over Lattices.** For a Gaussian width parameter  $\sigma \in \mathbb{R}^+$  and dimension  $n \in \mathbb{N}$ , let  $\rho_\sigma : \mathbb{R}^n \rightarrow \mathbb{R}$  be the Gaussian function:

$$\rho_\sigma(\mathbf{x}) = \exp \left( -\pi \|\mathbf{x}\|_2^2 / \sigma^2 \right).$$

For a coset  $\mathbf{c} + \Lambda$ , the Gaussian probability mass is defined as

$$\rho_\sigma(\mathbf{c} + \Lambda) = \sum_{\mathbf{x} \in \Lambda} \rho_\sigma(\mathbf{c} + \mathbf{x}).$$

We denote the discrete Gaussian distribution over  $\mathbb{Z}^n$  with parameter  $\sigma$  by

$$\mathcal{D}_{\mathbb{Z}^n, \sigma}(\mathbf{x}) = \frac{\rho_\sigma(\mathbf{x})}{\rho_\sigma(\mathbb{Z}^n)}.$$

For a matrix  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  and a vector  $\mathbf{v} \in \mathbb{Z}_q^n$ , we define  $\mathbf{A}_\sigma^{-1}(\mathbf{v})$  as the random variable  $\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma}$  conditioned on  $\mathbf{A}\mathbf{x} = \mathbf{v} \bmod q$ . This function extends naturally to matrices by applying  $\mathbf{A}_\sigma^{-1}$  to each column individually.

**Lemma 2 (From [19], Adapted from Lemma 8 of [2]).** *Let  $q \geq 2$  and let  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  be a matrix with  $m > n$ . Then for  $\mathbf{c} \in \mathbb{R}^m$  and  $\mathbf{u} \in \mathbb{Z}_q^n$ , it holds that*

$$\Pr\left[\|\mathbf{x}\| > \sqrt{m}\sigma : \mathbf{x} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{u})\right] \leq \text{negl}(n).$$

**Lemma 3 (Corollary 5.4 of [31]).** *Let  $n$  and prime  $q$  be positive integers, and  $m \geq 2n \log(q)$ . Then, for all but a  $\text{negl}(n)$  fraction of  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ , and for any  $\sigma \geq \omega(\sqrt{\log(m)})$ , the distribution of  $\mathbf{A}\mathbf{e} \bmod q$  where  $\mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}_q^m, \sigma}$  is statistically close to uniform over  $\mathbb{Z}_q^n$ .*

**Lemma 4 (Discrete Gaussian Preimages, a variant of Lemma 3.6 of [46]).** *Let  $n, m, q, t$  be parameters such that  $m \geq n$ . For any  $\ell, k = \text{poly}(n, \log(q))$ , consider the matrices  $\mathbf{A}_1, \dots, \mathbf{A}_\ell, \tilde{\mathbf{A}}_1, \dots, \tilde{\mathbf{A}}_\ell \in \mathbb{Z}_q^{n \times m}$ , the matrices  $\mathbf{B}_1, \dots, \mathbf{B}_\ell \in \mathbb{Z}_q^{n \times k}$  and a target vector  $\mathbf{t} \in \mathbb{Z}_q^{n\ell}$ . Here, both  $\mathbf{A}_i$  and  $\tilde{\mathbf{A}}_i$  are full rank, and  $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$  for all  $i \in [\ell]$ . Define the matrix:*

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 & & \mathbf{B}_1 \tilde{\mathbf{A}}_1 & & \\ & \mathbf{A}_2 & \mathbf{B}_2 \tilde{\mathbf{A}}_2 & & \\ & & \dots & \dots & \\ & & & \mathbf{A}_\ell \mathbf{B}_\ell & \tilde{\mathbf{A}}_\ell \end{bmatrix} \quad \text{and} \quad \mathbf{t} = \begin{bmatrix} \mathbf{t}_1 \\ \mathbf{t}_2 \\ \dots \\ \mathbf{t}_\ell \end{bmatrix}$$

*For all width parameters  $\sigma_1 \geq (q/\sigma_0) \log(2\ell m)$ , the statistical distance between the following distributions is negligible in  $m$ :*

$$\{\mathbf{v} : \mathbf{v} \leftarrow \mathbf{A}_{\sigma_1}^{-1}(\mathbf{t})\} \quad \text{and} \quad \left\{ \text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell, \hat{\mathbf{c}}, \tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_\ell) : \begin{matrix} \hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_1}^k \\ \text{col}(\mathbf{v}_i, \tilde{\mathbf{v}}_i) \leftarrow [\mathbf{A}_i | \tilde{\mathbf{A}}_i]_{\sigma_1}^{-1}(\mathbf{t}_i - \mathbf{B}_i \hat{\mathbf{c}}) \end{matrix} \right\}$$

When  $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$ , and  $\mathbf{A}_i$  and  $\tilde{\mathbf{A}}_i$  are full rank for each  $i \in [\ell]$ , it holds that  $\lambda_1^\infty(\text{diag}([\mathbf{A}_1 | \tilde{\mathbf{A}}_1], \dots, [\mathbf{A}_\ell | \tilde{\mathbf{A}}_\ell])) \geq \sigma_0$ , and  $\text{diag}([\mathbf{A}_1 | \tilde{\mathbf{A}}_1], \dots, [\mathbf{A}_\ell | \tilde{\mathbf{A}}_\ell])$  is full rank. Then, Lemma 4 follows from [48, Lemma 2.10].

**Lemma 5 (Lemma 5.1 of [31]).** *Let  $m \geq 2n \log(q)$ . Then, for all but at most a  $q^{-n}$  fraction of the matrices  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ , the subset-sums of the columns of  $\mathbf{A}$  generate  $\mathbb{Z}_q^n$ ; i.e., for every vector  $\mathbf{u} \in \mathbb{Z}_q^n$ , there exists an  $\mathbf{e} \in \{0, 1\}^m$  such that  $\mathbf{A}\mathbf{e} = \mathbf{u} \bmod q$ .*

**Lemma 6 (Adapted from Lemma 5.3 of [31]).** *Let  $n$  and prime  $q$  be positive integers, and let  $m \geq 2n \log(q)$ . Then, for all but at most a  $q^{-n}$  fraction of the matrices  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ , it holds that  $\lambda_1^\infty(\mathbf{A}) \geq \frac{q}{4}$ .*

**3.2.2 The Gadget Matrix and Trapdoors.** For  $n, q \in \mathbb{N}$ , the gadget matrix  $\mathbf{G}_n$  is defined as follows [35]:

$$\mathbf{G}_n = \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times m'},$$

where  $\mathbf{G}^\top = [1, 2, \dots, 2^{\lfloor \log(q) \rfloor}]$  and  $m' = n(\lfloor \log(q) \rfloor + 1)$ .

**Lemma 7 (Adapted from [35]).** *For any integers  $q \geq 2$ ,  $n \geq 1$  and  $m \geq n \lceil \log(q) \rceil$ , the gadget matrix  $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$  has a known basis  $\mathbf{S} \in \mathbb{Z}^{m \times m}$  with  $\|\mathbf{S}\|_{\text{GS}} \leq \sqrt{5}$  and  $\|\mathbf{S}\| \leq \max\{\sqrt{5}, \sqrt{k}\}$ , and preimage sampling for  $f_G(\mathbf{x}) = \mathbf{G}\mathbf{x} \bmod q$  with Gaussian parameter  $\sigma \geq \|\mathbf{S}\|_{\text{GS}} \cdot \omega(\sqrt{\log(n)})$  can be performed in time  $O(n \cdot \log^c(n))$  for some constant  $c$ .*

**Lemma 8 ([31]).** *Let  $n$  be a positive integer and  $q$  be a prime. Then, for any  $\sigma \geq \omega(\sqrt{\log(m')})$ , the distribution of  $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$ , where  $\mathbf{v} \leftarrow \mathbb{Z}_q^n$ , is statistically close to  $\mathcal{D}_{\mathbb{Z}_q, \sigma}^{m'}$ , where  $m' = n(\lfloor \log(q) \rfloor + 1)$ .*

Lemma 8 follows from the proof of [31, Corollary 5.4], along with [31, Lemma 3.1] and [35, Theorem 4.1] (cf. Appendix A for a longer discussion).

**Lemma 9 (Adapted from [35], Theorem 3.9 of [46]).** *Let  $n, m, q$  be lattice parameters and  $m' = n(\lfloor \log(q) \rfloor + 1)$ . Then, there exist efficient algorithms (TrapGen, SamplePre) such that;*

- **TrapGen**( $1^n, q, m$ )  $\rightarrow (\mathbf{A}, \mathbf{R})$ : *On input the lattice dimension  $n$ , the modulus  $q$  and the number of samples  $m$ , the trapdoor-generation algorithm outputs a matrix  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  together with a trapdoor  $\mathbf{R} \in \mathbb{Z}_q^{m \times m'}$ .*
- **SamplePre**( $\mathbf{A}, \mathbf{R}, \mathbf{v}, \sigma$ )  $\rightarrow \mathbf{u}$ : *On input a matrix  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ , a trapdoor  $\mathbf{R} \in \mathbb{Z}_q^{m \times m'}$ , a target vector  $\mathbf{v} \in \mathbb{Z}_q^n$  and a Gaussian width parameter  $\sigma$ , the preimage-sampling algorithm outputs a vector  $\mathbf{u} \in \mathbb{Z}_q^m$ .*

Moreover, for all  $m \geq O(n \log(q))$ , the above algorithms satisfy the following properties:

- **Trapdoor distribution:** *The matrix  $\mathbf{A}$  output by TrapGen( $1^n, q, m$ ) is statistically close to uniform. Specifically, if  $(\mathbf{A}, \mathbf{R}) \leftarrow \text{TrapGen}(1^n, q, m)$  and  $\mathbf{A}' \xleftarrow{r} \mathbb{Z}_q^{n \times m}$ , then  $\Delta(\mathbf{A}, \mathbf{A}') \leq 2^{-n}$ . Moreover,  $\mathbf{A}\mathbf{R} = \mathbf{G}$  and  $\|\mathbf{R}\|_\infty = 1$ .*
- **Preimage sampling:** *For all matrices  $\mathbf{R} \in \mathbb{Z}_q^{m \times m'}$ , parameters  $\sigma > 0$  and all target vectors  $\mathbf{v} \in \mathbb{Z}_q^n$  in the column span of  $\mathbf{A}$ , the output  $\mathbf{u} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{R}, \mathbf{v}, \sigma)$  satisfies  $\mathbf{A}\mathbf{u} = \mathbf{v}$ .*
- **Preimage distribution:** *Suppose  $\mathbf{R}$  is a gadget trapdoor for  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  (i.e.,  $\mathbf{A}\mathbf{R} = \mathbf{G}$ ). Then, for all  $\sigma \geq \sqrt{mm'} \|\mathbf{R}\|_\infty \cdot \omega(\sqrt{\log(n)})$  and all target vectors  $\mathbf{v} \in \mathbb{Z}_q^n$ , the statistical distance between the following distributions is at most  $2^{-n}$ :  $\{\mathbf{u} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{R}, \mathbf{v}, \sigma)\}$  and  $\{\mathbf{u} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{v})\}$ .*

For a matrix  $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_\ell] \in \mathbb{Z}_q^{n \times \ell}$ , we define  $\text{SamplePre}(\mathbf{A}, \mathbf{R}, \mathbf{V}, \sigma)$  as the algorithm that outputs the matrix where the  $i$ -th column is  $\text{SamplePre}(\mathbf{A}, \mathbf{R}, \mathbf{v}_i, \sigma)$ .

**3.2.3 Sampling Algorithms.** Let  $\mathbf{A}$  and  $\mathbf{B}$  denote matrices in  $\mathbb{Z}_q^{n \times m}$  and  $\mathbf{R}$  a matrix in  $\{-1, 1\}^{m \times m}$ . The BD scheme in Section 4 requires sampling short vectors statistically close to  $\mathbf{F}_\sigma^{-1}(\mathbf{v})$  for some vector  $\mathbf{v} \in \mathbb{Z}_q^n$  and matrix  $\mathbf{F} = [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}] \in \mathbb{Z}_q^{n \times 2m}$ . For this purpose, we use the following two algorithms from [2, §4.1] that enable sampling a preimage vector using either a trapdoor for  $\mathbf{A}$  or a trapdoor for  $\mathbf{B}$ :

1.  $\text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{v}, \sigma)$  takes a basis  $\mathbf{T}_\mathbf{A}$  for  $\mathbf{A}$ , a matrix  $\mathbf{M}_1$  and a Gaussian width parameter  $\sigma$  as input, and outputs a short vector  $\mathbf{e} \in \Lambda^\mathbf{v}(\mathbf{F})$  from a distribution statistically close to  $\mathbf{F}_\sigma^{-1}(\mathbf{v})$ , where  $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$ .
2.  $\text{SampleRight}(\mathbf{A}, \mathbf{B}, \mathbf{R}, \mathbf{T}_\mathbf{B}, \mathbf{u}, \sigma)$  takes a basis  $\mathbf{T}_\mathbf{B}$  for  $\mathbf{B}$ , a uniformly-random matrix  $\mathbf{R} \in \{-1, 1\}^{m \times m}$  and a Gaussian width  $\sigma$ , and outputs a short vector  $\mathbf{e} \in \Lambda^\mathbf{v}(\mathbf{F})$  from a distribution statistically close to  $\mathbf{F}_\sigma^{-1}(\mathbf{v})$ , where  $\mathbf{F} = [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}]$ .

**Lemma 10 (Lemma 17 of [2]).** *Given  $q > 2$ ,  $m > n$  and  $\sigma > \|\mathbf{T}_\mathbf{A}\|_{\text{GS}} \cdot \omega(\log(m + m_1))$ , output of  $\text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$  is distributed statistically close to  $\mathbf{F}_\sigma^{-1}(\mathbf{u})$ , where  $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$ .*

**Lemma 11 (Lemma 19 of [2]).** *Given  $q > 2$ ,  $m > n$  and  $\sigma > \|\mathbf{T}_\mathbf{B}\|_{\text{GS}} \cdot s_\mathbf{R} \cdot \omega(\sqrt{\log(m)})$ , output of  $\text{SampleRight}(\mathbf{A}, \mathbf{B}, \mathbf{R}, \mathbf{T}_\mathbf{B}, \mathbf{u}, \sigma)$  is distributed statistically close to  $\mathbf{F}_\sigma^{-1}(\mathbf{u})$ , where  $s_\mathbf{R} := \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$ , and  $\mathbf{F} = [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}]$ .*

**3.2.4 Leftover Hash Lemma.** The following lemma follows from a generalization of the leftover hash lemma [27].

**Lemma 12 (Lemma 13 of [2], from [27]).** *Suppose that  $m \geq (n + 1) \log(q) + \omega(\log(n))$  and  $q > 2$  is prime. Let  $\mathbf{R}$  be an  $(m \times k)$  uniformly-random matrix in  $\{1, -1\}^{m \times k} \bmod q$ , where  $k = k(n)$  is a polynomial in  $n$ . Let  $\mathbf{A}$  and  $\mathbf{B}$  be uniformly-random matrices in  $\mathbb{Z}_q^{n \times m}$  and  $\mathbb{Z}_q^{n \times k}$ , respectively. Then, for all vectors  $\mathbf{w} \in \mathbb{Z}_q^m$ , the distribution  $(\mathbf{A}, \mathbf{A}\mathbf{R}, \mathbf{R}^\top \mathbf{w})$  is statistically close to the distribution  $(\mathbf{A}, \mathbf{B}, \mathbf{R}^\top \mathbf{w})$ .*

Let  $\Psi_\alpha$  denote the distribution over  $\mathbb{Z}_q$  of the random variable  $[qX] \bmod q$ , where  $X$  is a normal random variable with mean 0 and standard deviation  $\alpha/\sqrt{2\pi}$ .

**Lemma 13 (Lemma 12 of [2]).** *Let  $\mathbf{e} \in \mathbb{Z}^m$  be some vector and let  $\mathbf{y} \in [0, q - 1]^m$  be an integer vector such that  $\mathbf{y} \leftarrow \Psi_\alpha^m$ . Then the quantity  $|\mathbf{e}^\top \mathbf{y}|$ , treated as an integer, satisfies the following except with probability  $\text{negl}(m)$ :*

$$|\mathbf{e}^\top \mathbf{y}| \leq \|\mathbf{e}\| q \alpha \cdot \omega(\log(m)) + \|\mathbf{e}\| \frac{\sqrt{m}}{2}$$

The learning with errors problem is formally defined in Appendix A.

### 3.3 Shifted Multi-Preimage Trapdoor Sampler

Our BD scheme makes use of the following shifted multi-preimage trapdoor sampler, introduced and instantiated by Waters et al. [46].

**Definition 4 (Definitions 4.2 and 4.4 from [46]).** *Let  $\ell$  be a dimension parameter, and  $n, t, q, \sigma$  be functions of  $\lambda$  and  $\ell$ . An  $(n, t, q, \sigma)$ -shifted multi-preimage trapdoor sampler is a tuple of efficient algorithms with the following syntax:*

- $\text{Gen}(1^\lambda, 1^\ell) \rightarrow \text{crs}$ : The generation algorithm outputs a common reference string  $\text{crs}$ .
- $\text{Expand}(1^\lambda, 1^\ell, \text{crs}, \{u_i\}_{i \in [\ell]}) \rightarrow (\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td})$ : The deterministic expand algorithm takes  $\ell$  distinct bit vectors  $\{u_i\}_{i \in [\ell]}$ , outputs matrices  $\mathbf{A}_1, \dots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times t}$  and a matrix-trapdoor tuple  $\text{td}$ .
- $\text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell) \rightarrow (\pi_1, \dots, \pi_\ell, \mathbf{c})$ : Given a matrix-trapdoor tuple  $\text{td}$  and target vectors  $\mathbf{t}_1, \dots, \mathbf{t}_\ell$ , the sampling algorithm outputs a shift  $\mathbf{c} \in \mathbb{Z}_q^n$  and the preimages  $\pi_i \in \mathbb{Z}_q^t$ .

The sampler must satisfy:

- **Correctness:** For all  $\lambda, \ell \in \mathbb{N}$ , all crs in the support of  $\text{Gen}(1^\lambda, 1^\ell)$ , and all  $\mathbf{t}_1, \dots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$ , given  $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs})$ , it holds that

$$\Pr[\mathbf{A}_i \pi_i = \mathbf{t}_i + \mathbf{c} \ \forall i \in [\ell]: (\pi_1, \dots, \pi_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell)] = 1.$$

- **Preimage distribution:** For all polynomials  $\ell = \ell(\lambda)$ , except with negligible probability over  $\text{crs} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$ , and  $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs})$ , the statistical distance between the following distributions is  $\text{negl}(\lambda)$ :

$$\{(\pi_1, \dots, \pi_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell)\} \text{ and } \left\{(\pi_1, \dots, \pi_\ell, \mathbf{c}) \left| \begin{array}{c} \mathbf{c} \leftarrow \mathbb{Z}_q^n, \\ \pi_i \leftarrow (\mathbf{A}_i^{-1})_\sigma(\mathbf{t}_i + \mathbf{c}) \ \forall i \in [\ell] \end{array} \right. \right\}$$

- **Somewhere programmable:** There exists an efficient algorithm  $\text{GenProg}$  such that for all polynomials  $\ell = \ell(\lambda)$ , and  $\ell$  distinct bit vectors  $\{u_i\}_{i \in [\ell]}$ , the following holds:
  - $\text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i)$  takes a bit vector  $u_i$  and matrix  $\mathbf{A}_i$ , and outputs a common reference string  $\tilde{\text{crs}}$ .
  - $\forall i \in [\ell], \forall \mathbf{A}_i \in \mathbb{Z}_q^{n \times t}$ :

$$\Pr\left[\mathbf{A}_i = \tilde{\mathbf{A}}_i \left| \begin{array}{c} \tilde{\text{crs}} \leftarrow \text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i), \\ (\tilde{\mathbf{A}}_1, \dots, \tilde{\mathbf{A}}_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \tilde{\text{crs}}, \{u_i\}_{i \in [\ell]}) \end{array} \right. \right] = 1.$$

- The statistical distance between the following distributions is  $\text{negl}(\lambda)$ :

$$\{\text{crs} \leftarrow \text{Gen}(1^\lambda, 1^\ell)\} \text{ and } \left\{\tilde{\text{crs}} \left| \begin{array}{c} \mathbf{A}_i \leftarrow \mathbb{Z}_q^{n \times t}, \\ \tilde{\text{crs}} \leftarrow \text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i) \end{array} \right. \right\}$$

- **Local expansion:** There exists an efficient deterministic algorithm  $\text{ExpandLocal}$  such that:
  - $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i)$  takes a bit vector  $u_i$  and outputs  $\mathbf{A}_i \in \mathbb{Z}_q^{n \times t}$ .
  - For all  $\lambda, \ell \in \mathbb{N}$  and all crs, given

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}, \{u_i\}_{i \in [\ell]}),$$

it holds that for all  $i \in [\ell]$ ,  $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i) = \mathbf{A}_i$ . Moreover,  $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i)$  only reads  $\text{poly}(\lambda, \log(\ell))$  bits of crs and runs in time  $\text{poly}(\lambda, \log(\ell))$ .

**3.3.1 Shifted Multi-Preimage Trapdoor Sampler Construction.** We next describe the shifted multi-preimage trapdoor sampler construction by Waters et al. [46, Construction 4.6]. Let  $n = n(\lambda, \ell)$ ,  $q = q(\lambda, \ell)$ ,  $\sigma = \sigma(\lambda, \ell)$ ,  $m = 3n \lceil \log q \rceil$  and  $t = m(\lceil \log \ell \rceil + 1)$ . The vectors  $u_i$  are drawn from the set  $\{0, 1\}^{\lceil \log \ell \rceil}$ .

- $\text{Gen}(1^\lambda, 1^\ell)$ : The algorithm samples  $[\mathbf{A} \mid \mathbf{B}] \leftarrow \mathbb{Z}_q^{n \times t}$ , where  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  and  $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$ , and outputs  $\text{crs} = [\mathbf{A} \mid \mathbf{B}]$ .
- $\text{Expand}(1^\lambda, 1^\ell, \text{crs}, \{u_i\}_{i \in [\ell]})$ : Given  $\text{crs} = [\mathbf{A} \mid \mathbf{B}]$ , where  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  and  $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$ , the expand algorithm does the following:



- For each  $i \in [\ell]$ , let  $\mathbf{B}_i := \mathbf{B} - u_i^\top \otimes \mathbf{G}$ , and

$$\mathbf{A}_i := [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] = [\mathbf{A} \mid \mathbf{B}_i] \in \mathbb{Z}_q^{n \times t}.$$

- Let  $\mathbf{H} \in \{0, 1\}^{(\ell t + m) \times (\ell t + m)}$  be the permutation matrix such that:

$$\begin{aligned} \mathbf{D}_\ell &:= \begin{bmatrix} \mathbf{A} & \mathbf{B}_1 & & & \mathbf{G} \\ & \mathbf{A} & \mathbf{B}_2 & & \mathbf{G} \\ & & \dots & & \mathbf{G} \\ & & & \mathbf{A} & \mathbf{B}_\ell & \mathbf{G} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A} & & & \mathbf{B}_1 & & \mathbf{G} \\ & \dots & & & \dots & \mathbf{G} \\ & & \dots & & \dots & \mathbf{G} \\ & & & \mathbf{A} & & \mathbf{B}_\ell & \mathbf{G} \end{bmatrix} \mathbf{H} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)} \end{aligned}$$

- Compute  $\mathbf{T}' \leftarrow \text{StructTrapGen}(\mathbf{B}, \{u_i\}_{i \in [\ell]}) \in \mathbb{Z}_q^{(\ell m \lceil \log \ell \rceil + m) \times \ell m}$ , where **StructTrapGen** is the algorithm given in [46, Lemma 4.1], and outputs a trapdoor  $\mathbf{T}'$  for the following matrix such that  $\|\mathbf{T}'\|_\infty = 1$ :

$$\begin{bmatrix} \mathbf{B} - u_1 \otimes \mathbf{G} & & & \mathbf{G} \\ & \mathbf{B} - u_2 \otimes \mathbf{G} & & \mathbf{G} \\ & & \dots & \mathbf{G} \\ & & & \mathbf{B} - u_\ell \otimes \mathbf{G} & \mathbf{G} \end{bmatrix}$$

Finally, define the trapdoor  $\mathbf{T} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$  as:

$$\mathbf{T} := \mathbf{H}^{-1} \begin{bmatrix} 0_{\ell m \times \ell m} \\ \mathbf{T}' \end{bmatrix}.$$

The algorithm sets  $\text{td} := (\mathbf{D}_\ell, \mathbf{T})$  and outputs  $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td})$ .

- **SampleMultPre**( $\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell$ ): Given the trapdoor  $\text{td} = (\mathbf{D}_\ell, \mathbf{T})$  and the target vectors  $\mathbf{t}_1, \dots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$ , the shifted multi-preimage sampler defines the vector  $\mathbf{t} := \text{col}(\mathbf{t}_1, \dots, \mathbf{t}_\ell)$  and outputs  $(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, -\mathbf{G}\hat{\mathbf{c}})$ , obtained using **SamplePre**:  $\text{col}(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, \hat{\mathbf{c}}) \leftarrow \text{SamplePre}(\mathbf{D}_\ell, \mathbf{T}, \mathbf{t}, \sigma)$ .
- **GenProg**( $1^\lambda, 1^\ell, u_i, \mathbf{A}_i$ ): Given the vector  $u_i \in \{0, 1\}^{\lceil \log(\ell) \rceil}$  and the matrix  $\mathbf{A}_i = [\mathbf{A} \mid \mathbf{B}]$ , where  $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$  and  $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$ , the algorithm outputs  $\text{crs} = [\mathbf{A} \mid \mathbf{B} + u_i^\top \otimes \mathbf{G}]$ .
- **ExpandLocal**( $1^\lambda, \text{crs}, u_i$ ): Given the  $\text{crs} = [\mathbf{A} \mid \mathbf{B}]$  and the vector  $u_i \in \{0, 1\}^{\lceil \log(\ell) \rceil}$ , the algorithm outputs  $\mathbf{A}_i = [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] \in \mathbb{Z}_q^{n \times t}$ .

**Lemma 14 (Adapted from Theorem 4.7 of [46]).** *Let  $n \geq \lambda$ ,  $n = n(\lambda, \ell)$ ,  $q = q(\lambda, \ell)$ ,  $m = 3n \lceil \log q \rceil$ , and  $t = m(\lceil \log \ell \rceil + 1)$ . Then for all  $\sigma \geq (\ell t + m) \log(\ell n)$ , Construction 4.6 by Waters et al. [46] is an  $(n, t, q, \sigma)$ -shifted multi-preimage trapdoor sampler with (perfect) somewhere programmability and local expansion. The size of the CRS output by  $\text{Gen}(1^\lambda, 1^\ell)$  is  $nt \log(q)$ .*

## 4 Our Batch Decryption System

In this section, we present our full epochless batch decryption (BD) scheme. The scheme uses the parameter  $\ell$  that denotes the maximum batch size, and  $q, n, m, t, \beta, \sigma_1, \sigma_2$  specified in Section 4.1. It also makes use of the following hash functions:

**Definition 5 (Random Oracle  $H$ ).** The function  $H: \mathcal{I} \rightarrow \mathbb{Z}_q^n \times \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$  maps identities to vectors. Upon receiving a new input  $r \in \mathcal{I}$ , it samples  $\mathbf{h} \leftarrow \mathbb{Z}_q^n$  and  $\tilde{h} \leftarrow \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ , and outputs  $(\mathbf{h}, \tilde{h})$ .

**Definition 6 (Random Oracle  $H_{\text{sp}}$ ).** The function

$$H_{\text{sp}}: \mathbb{Z}_q^{\ell n \times (\ell t + m)} \times \mathbb{Z}_q^{\ell n \times \ell t} \times \mathbb{Z}_q^{(\ell t + m) \times \ell m} \times \mathbb{Z}_q^{\ell n} \times \mathbb{R} \rightarrow \mathbb{Z}_q^{2\ell t + m}$$

takes as input matrices  $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}$ ,  $\hat{\mathbf{A}} \in \mathbb{Z}_q^{\ell n \times \ell t}$ , a trapdoor  $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$  for the matrix  $\mathbf{A}$ , a target vector  $\mathbf{h} \in \mathbb{Z}_q^{\ell n}$  and a Gaussian width  $\sigma$ . It outputs a sample from  $\text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma)$  using  $\text{SampleLeft}$  from Section 3.2.

**Definition 7 (Hash Function  $H_{\text{frd}}$ , Section 5 in [2]).** Given a prime  $q$  and an integer  $n \in \mathbb{Z}^+$ , the encoding with full-rank differences,  $H_{\text{frd}}: \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^{n \times n}$ , ensures that for all  $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_q^n$ ,  $\mathbf{u} \neq \mathbf{v}$ , the matrix  $H(\mathbf{u}) - H(\mathbf{v}) \in \mathbb{Z}_q^{n \times n}$  is full-rank.

We can now describe the algorithms of our batch decryption scheme. Fix some Gaussian width parameters  $\sigma_1$  and  $\sigma_2$ . Let

$$\Pi_{\text{samp}} = (\text{Gen}, \text{GenTD}, \text{Expand}, \text{ExpandLocal})$$

denote the  $(n, t, q, \sigma_1)$ -shifted multi-preimage trapdoor sampler with local expansion from [46, Construction 4.6] (cf. Section 3.3).

**BD.Setup** $(1^\lambda, 1^\ell)$ . The setup algorithm first calculates  $\text{crs}_{\text{samp}} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$ . It then samples two uniformly-random matrices  $\tilde{\mathbf{A}} \in \mathbb{Z}_q^{n \times t}$  and  $\tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times t}$ , as well as a tuple  $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \text{TrapGen}(1^n, q, m)$ . Finally, the algorithm outputs

$$\text{pk} \leftarrow (\text{crs}_{\text{samp}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \mathbf{C}) \quad \text{sk} \leftarrow (\text{pk}, \mathbf{T}_{\mathbf{C}}).$$

**BD.Enc** $(\text{pk}, r \in \mathcal{I}, \mathbf{m} \in \{0, 1\}^{\beta \lceil \log(\ell) \rceil})$ . The encryption algorithm first samples a uniformly-random vector  $\mathbf{s}_r \leftarrow \mathbb{Z}_q^n$  and calculates  $H(r) = (\mathbf{h}_r, \tilde{h}_r) \in \mathbb{Z}_q^n \times \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ . It then computes  $\mathbf{A}_r = \text{ExpandLocal}(1^\lambda, \text{crs}_{\text{samp}}, \tilde{h}_r) \in \mathbb{Z}_q^{n \times t}$ . It next selects the noise vectors  $\boldsymbol{\epsilon}_{1,r} \leftarrow \Psi_\alpha$ ,  $\boldsymbol{\epsilon}_{2,r} \leftarrow \Psi_\alpha^m$ , and  $\boldsymbol{\epsilon}_{3,r,\text{pre}} \leftarrow \Psi_\alpha^t$ , along with a matrix  $\tilde{\mathbf{R}}_r \leftarrow \{-1, 1\}^{t \times t}$ . Finally, it sets  $\boldsymbol{\epsilon}_{3,r} \leftarrow \text{col}(\boldsymbol{\epsilon}_{3,r,\text{pre}}, \tilde{\mathbf{R}}_r^\top \boldsymbol{\epsilon}_{3,r,\text{pre}})$ ,  $\mathbf{F}_r \leftarrow [\mathbf{A}_r | \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_r) \tilde{\mathbf{B}}] \in \mathbb{Z}_q^{n \times 2t}$ , and finds the ciphertext  $\text{ct}_r = (r, \text{ct}_r[1], \text{ct}_r[2], \text{ct}_r[3])$  as follows:

$$\begin{aligned} \text{ct}_r[1] &:= \mathbf{h}_r^\top \mathbf{s}_r + \boldsymbol{\epsilon}_{1,r} + \mathbf{m} \cdot \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}_r[2] &:= \mathbf{C}^\top \mathbf{s}_r + \boldsymbol{\epsilon}_{2,r} \in \mathbb{Z}_q^m \\ \text{ct}_r[3] &:= \mathbf{F}_r^\top \mathbf{s}_r + \boldsymbol{\epsilon}_{3,r} \in \mathbb{Z}_q^{2t} \end{aligned}$$

**BD.PreDec** $(\text{sk}, (r_i)_{i \in [\ell]})$ . The pre-decryption algorithm first calculates  $H(r_i) = (\mathbf{h}_i, \tilde{h}_i) \in \mathbb{Z}_q^n \times \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$  for each  $i \in [\ell]$ . It requires that  $\tilde{h}_i$  are distinct vectors for each  $i \in [\ell]$  (otherwise, it aborts). Here,  $r_i$  can be ordered in any predetermined order. It then generates

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td} = (\mathbf{A}, \mathbf{T}_{\mathbf{A}})) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, \{\tilde{h}_i\}_{i \in [\ell]}),$$

where

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_1 & & -\mathbf{G} \\ & \mathbf{A}_2 & -\mathbf{G} \\ & & \dots \\ & & & \mathbf{A}_\ell & -\mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}, \quad (1)$$

and constructs

$$\hat{\mathbf{A}} = \begin{bmatrix} \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_1)\tilde{\mathbf{B}} & & & \\ & \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_2)\tilde{\mathbf{B}} & & \\ & & \dots & \\ & & & \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_\ell)\tilde{\mathbf{B}} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times \ell t} \quad (2)$$

Using these matrices, it samples

$$\text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) \leftarrow H_{\text{sp}}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell), \sigma_1),$$

setting  $\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$  for each  $i \in [\ell]$ , and  $\mathbf{c} := \mathbf{G}\hat{\mathbf{c}}$ . Finally, it samples and outputs the following vector as the pre-decryption key:

$$\mathbf{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_\mathbf{C}, \mathbf{c}, \sigma_2) \in \mathbb{Z}_q^m$$

**BD.Dec**( $\text{pk}, \mathbf{sbk}, (\text{ct}_i)_{i \in [\ell]}$ ). The decryption algorithm first parses the ciphertexts:  $(r_i, \text{ct}_i[1], \text{ct}_i[2], \text{ct}_i[3]) \leftarrow \text{ct}_i$ . Computing  $H(r_i) = (\mathbf{h}_i, \tilde{\mathbf{h}}_i) \in \mathbb{Z}_q^n \times \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$  for  $i \in [\ell]$ , where  $\tilde{\mathbf{h}}_i$  are distinct binary vectors, it generates

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \mathbf{T}_\mathbf{A}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, \{\tilde{\mathbf{h}}_i\}_{i \in [\ell]})$$

and constructs the matrices  $\mathbf{A}$  and  $\hat{\mathbf{A}}$  as in equations (1) and (2). It then samples

$$\text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) \leftarrow H_{\text{sp}}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell), \sigma_1),$$

and sets  $\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$  for each  $i \in [\ell]$ , and  $\mathbf{c} := \mathbf{G}\hat{\mathbf{c}}$ . Finally, for each  $i \in [\ell]$ , it outputs  $\mathbf{m}'_i$  as the message:

$$\mathbf{m}'_i \leftarrow \text{round} \left( \text{ct}_i[1] - \mathbf{sbk}^\top \text{ct}_i[2] + \mathbf{v}_i^\top \text{ct}_i[3] \right).$$

Here,  $\text{round}(x)$  outputs 1 if  $|x - \lfloor q/2 \rfloor| < \lfloor q/4 \rfloor$ , and 0 otherwise.

*Remark 2 (The role of  $\beta$ ).* The parameter  $\beta \in \mathbb{Z}^+$  determines the size of the vectors  $\tilde{\mathbf{h}}_i$ . In other words,  $\beta \lceil \log(\ell) \rceil$  is the logarithm of the size of the identity space. Therefore, larger  $\beta$  implies smaller chance of collision on the vectors  $\tilde{\mathbf{h}}_i$  for different identities. Since pre-decryption keys can only be generated for batches of identities  $\{r_i\}_{i \in \ell}$  with distinct vectors  $\tilde{\mathbf{h}}_i$ , when the vectors  $\tilde{\mathbf{h}}_i$  are generated using the random oracle  $H$ ,  $\beta$  must be  $O(\lambda)$  to ensure that  $\tilde{\mathbf{h}}_i \neq \tilde{\mathbf{h}}_j$  for the identities  $r_i \neq r_j$  within a batch, except with negligible probability. To avoid this blow-up, it is possible for the encryptors to coordinate in advance, or use the same technique as in [18, §5.3]. This technique requires using multiple sub-batches, encrypting each message for every sub-batch and deriving pre-decryption keys for each sub-batch; so that the ciphertexts can be eventually grouped into sub-batches with distinct vectors  $\tilde{\mathbf{h}}_i$  and decrypted (cf., [18, §5.3] for numerical analysis). For instance, setting  $\beta = 1$  implies a collision of  $O(\log(\ell))$   $\tilde{\mathbf{h}}_i$  values with high probability, thus necessitating  $O(\log(\ell))$  sub-batches, and implying a  $O(\log(\ell))$  factor increase in the pre-decryption key and ciphertext size. In Section 4.4, we propose an alternative resolution, where we can achieve  $\beta = 1$  at the expense of a  $O(\log(\ell))$ -factor increase *only* in the ciphertext size.

#### 4.1 Correctness and Parameter Sizes

**Theorem 1.** *The BD scheme is correct, as in Definition 2, whenever*

$$\begin{aligned} n &\geq \lambda & m &\geq 3n \lceil \log q \rceil & t &= m(\beta \lceil \log \ell \rceil + 1) \\ \sigma_1 &\geq (\ell t + m) \cdot \omega(\log(2\ell t + m)) & \sigma_2 &\geq m \cdot \omega(\sqrt{\log(n)}) \\ q &= O(\ell t^{5/2}) \cdot \omega(\log(2\ell t + m) \log(t)) & \alpha &= O([\ell t^2 \cdot \omega(\log(2\ell t + m) \log(t))]^{-1}) \end{aligned}$$

where  $q$  is rounded up to the nearest larger prime.

*Proof.* Since  $n \geq \lambda$ ,  $q > 2$ ,  $m \geq 3n \lceil \log q \rceil$ ,  $t = m(\beta \lceil \log \ell \rceil + 1)$ ,  $\sigma_1 \geq (\ell t + m) \cdot \omega(\log(2\ell t + m))$  and  $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$ , by Lemma 22, we make the following observations:

1. Construction 4.6 by Waters et al. [46] (cf. Lemma 14) is an  $(n, t, q, \sigma_1)$ -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion. Therefore, by the local expansion property, for all  $\lambda, \ell \in \mathbb{N}$  and for all  $\text{crs}$ , given  $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}, \{\tilde{h}_i\}_{i \in [\ell]})$ , it holds that  $\text{ExpandLocal}(1^\lambda, \text{crs}, \tilde{h}_i) = \mathbf{A}_i$  for all  $i \in [\ell]$ .
2.  $m \geq O(n \log(q))$  and  $\sigma_2 \geq m \|\mathbf{T}_C\|_\infty \cdot \omega(\sqrt{\log(n)})$ . Then, the output of the algorithms `TrapGen` and `SamplePre` in `BD.SETUP` and `BD.PREDEC` satisfy Lemma 9.
3.  $\sigma_1 \geq \|\mathbf{T}_A\|_{\text{GS}} \cdot \omega(\log(2\ell t + m))$ . Hence, the output of the algorithm `SampleLeft` and the random oracle  $H_{\text{sp}}$  satisfy Lemma 10.
4. For all  $i \in [\ell]$  and  $\mathbf{h} \in \mathbb{Z}_q$ , the matrices  $\mathbf{A}_i$ ,  $\tilde{\mathbf{A}}$ ,  $\tilde{\mathbf{B}}$  and  $\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h})\tilde{\mathbf{B}}$  are full-rank (rank  $n$ ), and there exists a value  $w$  such that  $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h})\tilde{\mathbf{B}}) \geq w$ . Moreover,  $\sigma_1 \geq (q/w) \log(2\ell t)$ .

For a pre-decryption key  $\mathbf{sbk}$  and ciphertext  $\text{ct}_i$  generated via the algorithms `BD.PREDEC` and `BD.ENC`, by items (1)-(2)-(3) (i.e., the properties of the shifted multi-preimage trapdoor sampler, and Lemmas 9 and 10), it holds that  $\mathbf{F}_{r_i} \cdot \mathbf{v}_i + \mathbf{h}_i = \mathbf{C} \cdot \mathbf{sbk}$ , where  $\mathbf{F}_{r_i} = [\mathbf{A}_i | \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_i)\tilde{\mathbf{B}}]$ . Consequently, for  $i \in [\ell]$ ,

$$\begin{aligned} \text{ct}_i[1] &- \left( \mathbf{sbk}^\top \text{ct}_i[2] - \mathbf{v}_i^\top \text{ct}_i[3] \right) \\ &= \mathbf{h}_i^\top \mathbf{s}_i + \epsilon_{1,i} + \mathbf{m}_i \cdot \lfloor q/2 \rfloor \\ &\quad - \left( \mathbf{sbk}^\top \mathbf{C}^\top \mathbf{s}_i + \mathbf{sbk}^\top \epsilon_{2,i} - \mathbf{v}_i^\top \mathbf{F}_{r_i}^\top \mathbf{s}_i - \mathbf{v}_i^\top \epsilon_{3,i} \right) \\ &= (\mathbf{h}_i - \mathbf{C} \cdot \mathbf{sbk} + \mathbf{F}_{r_i} \cdot \mathbf{v}_i)^\top \mathbf{s}_i + \epsilon_{1,i} - \left( \mathbf{sbk}^\top \epsilon_{2,i} - \mathbf{v}_i^\top \epsilon_{3,i} \right) + \mathbf{m}_i \cdot \lfloor q/2 \rfloor \\ &= \left( \epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} + \mathbf{v}_i^\top \epsilon_{3,i} \right) + \mathbf{m}_i \cdot \lfloor q/2 \rfloor \end{aligned}$$

Recall that  $\epsilon_{3,i} = \text{col}(\epsilon_{3,i,\text{pre}}, \tilde{\mathbf{R}}_i^\top \epsilon_{3,i,\text{pre}})$ . Therefore,

$$\epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} + \mathbf{v}_i^\top \epsilon_{3,i} = \epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} + (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}$$

Next, we observe that by item (3) (i.e., Lemma 10), output of  $H_{\text{sp}}$  is statistically indistinguishable from a sample from  $\mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$ , where  $\mathbf{h} = \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell)$  and  $\mathbf{F} = [\mathbf{A} | \tilde{\mathbf{A}}]$  (cf. equations (1) and (2) for  $\mathbf{A}$  and  $\tilde{\mathbf{A}}$ ). Therefore, by item (4), we can invoke Lemma 4 for  $(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$ , and infer that  $\mathbf{v}_i$  is a Gaussian sample with width parameter  $\sigma_1$ . Combining this with Lemma 2, we observe that for all  $i \in [\ell]$ ,  $\|\mathbf{v}_i\| \leq \sqrt{2t}\sigma_1$  except with negligible probability. Then, by Lemma 1,  $\|\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq \|\mathbf{v}_i^1\| + \|\tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq O(t\sigma_1)$ .

By item (2) (*i.e.*, Lemma 9), we know that  $\mathbf{sbk}$  is a Gaussian sample with width parameter  $\sigma_2$ . Again, combining this with Lemma 2, we observe that  $\|\mathbf{sbk}\| \leq \sqrt{m}\sigma_2$  except with negligible probability. Hence, by Lemma 13,

$$\begin{aligned} \|\epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} + (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}\| &\leq \\ &\leq |\epsilon_{1,i}| + |\mathbf{sbk}^\top \epsilon_{2,i}| + |(\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}| \\ &\leq (t\sigma_1 + \sqrt{m}\sigma_2)q\alpha \cdot \omega(\log(t)) + O\left((t\sigma_1 + \sqrt{m}\sigma_2) \cdot \frac{\sqrt{t}}{2}\right) \end{aligned}$$

Finally, for decryption to succeed, the error term must be less than  $q/5$  except with negligible probability. For this to be true, the inequality above requires that  $\alpha < ((t\sigma_1 + \sqrt{m}\sigma_2) \cdot \omega(\log(t)))^{-1}$  and  $q = \Omega(\sigma_1 t^{3/2} + \sigma_2 \sqrt{mt})$ , which are satisfied by the values of  $\alpha$  and  $q$ .  $\square$

Given the size of the CRS, the public key scales as

$$|\mathbf{pk}| = n(4t + m) \log(q) = O(\beta \lambda^2 \log^2(\lambda) \log(\ell) + \beta \lambda^2 \log^3(\ell)),$$

the pre-decryption key scales as

$$|\mathbf{sbk}| = O(m \log(\sigma_2)) = O(\lambda \log^2(\lambda) + \lambda \log(\lambda) \log(\ell)),$$

and the ciphertexts scale as

$$|\mathbf{ct}| = (1 + m + 2t) \log(q) = O(\beta \lambda \log^2(\lambda) \log(\ell) + \beta \lambda \log^3(\ell))$$

as a function of  $\lambda$ ,  $\log(\ell)$  and  $\beta$ . As  $\beta \log(\ell)$  denotes the logarithm of the size of the identity space, in Table 1,  $\beta \log(\ell)$  is replaced by  $k$ , which denotes the logarithm of the size of the identity space.

*Remark 3.* Among the matrices  $\text{crs}_{\text{samp}} = [\mathbf{A} \mid \mathbf{B}]$ ,  $\tilde{\mathbf{A}}$ ,  $\tilde{\mathbf{B}}$  and  $\mathbf{C}$  that constitute the public keys of our batch decryption system (see Section 3.3 for the structure of  $\text{crs}_{\text{samp}}$ ),  $\mathbf{A}$ ,  $\mathbf{B}$ ,  $\tilde{\mathbf{A}}$  and  $\tilde{\mathbf{B}}$  are randomly-generated. Thus, they can instead be generated through a random oracle evaluation. Then, the public key only includes the matrix  $\mathbf{C}$  of size

$$|\mathbf{pk}| = O(nm \log(q)) = O(n^2 \log^2(q)) = O(\lambda^2 \log^2(\ell) + \lambda^2 \log^2(\lambda)).$$

We prove security of the scheme as described at the beginning of Section 4, and note that a small modification proves security of the version with shorter public keys. The only change is that whenever the simulator chooses the matrices  $\mathbf{A}$ ,  $\mathbf{B}$ ,  $\tilde{\mathbf{A}}$  and  $\tilde{\mathbf{B}}$ , it will instead program the random oracle used to generate those matrices.

## 4.2 Security

**Theorem 2.** *The BD scheme is selectively secure (Definition 3), whenever*

$$\begin{aligned} n &\geq \lambda & m &\geq 3n \lceil \log q \rceil & t &= m(\beta \lceil \log \ell \rceil + 1) \\ \sigma_1 &\geq (\ell t + m) \cdot \omega(\log(2\ell t + m)) & \sigma_2 &\geq m \cdot \omega(\sqrt{\log(n)}) & q &> 2\sqrt{n}/\alpha, \end{aligned}$$

and the  $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE assumption holds, and  $H$  and  $H_{\text{sp}}$  are modeled as random oracles.

*Proof.* Let  $\mathcal{A}$  be a PPT adversary attacking BD that has advantage  $\epsilon$  using polynomial number of queries to  $H$  and  $H_{\text{st}}$ . Let  $r^*$  denote the identity to be attacked. Consider the following sequence of seven hybrid games between  $\mathcal{A}$  and the challenger Ch:

**Hybrid 0:** This is the game  $\text{Game}_{\mathcal{A},b}^{\text{BD}}(1^\lambda, \ell)$  in the random oracle model:

- **Before the setup**,  $\mathcal{A}$  sends a challenge identity  $r^*$  to Ch.
- **At the setup**, Ch calculates  $\text{crs}_{\text{samp}} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$ . It then samples two uniformly-random matrices  $\tilde{\mathbf{A}} \in \mathbb{Z}_q^{n \times t}$  and  $\tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times t}$ , as well as a tuple  $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \text{TrapGen}(1^n, q, m)$ . It sends  $\text{pk} := (\text{crs}_{\text{samp}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \mathbf{C})$  to  $\mathcal{A}$ .
- **Simulating random oracle queries to  $H$** : Upon receiving a new query  $r$ , Ch samples  $\mathbf{h}_r \leftarrow \mathbb{Z}_q^n$  and  $\tilde{h}_r \leftarrow \{0, 1\}^{\lceil \log(q) \rceil}$ , stores the tuple  $(r, \mathbf{h}_r, \tilde{h}_r)$ , and returns  $(\mathbf{h}_r, \tilde{h}_r)$  to  $\mathcal{A}$ .
- **Simulating random oracle queries to  $H_{\text{sp}}$** : Upon receiving a new input with the matrices  $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}$ ,  $\hat{\mathbf{A}} \in \mathbb{Z}_q^{\ell n \times \ell t}$ ,  $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$ , vector  $\mathbf{h} \in \mathbb{Z}_q^{\ell n}$  and Gaussian width  $\sigma$ , Ch samples a vector

$$\mathbf{v} \leftarrow \text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma),$$

stores the tuple  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma, \mathbf{v})$ , and returns  $\mathbf{v}$  to  $\mathcal{A}$ .

- **To respond to pre-challenge queries** given a list of identities  $(r_i)_{i \in [\ell]}$ , Ch runs the algorithm  $\text{BD.PREDEC}(\text{sk}, (r_i)_{i \in [\ell]})$ . For this purpose, it calculates  $H(r_i) = (\mathbf{h}_i, \tilde{h}_i)$  for  $i \in [\ell]$  and runs  $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td} = (\mathbf{A}, \mathbf{T}_{\mathbf{A}})) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, \{\tilde{h}_i\}_{i \in [\ell]})$  to obtain the matrices  $\mathbf{A}$ ,  $\hat{\mathbf{A}}$  and  $\mathbf{T}_{\mathbf{A}}$  (cf., equations (1) and (2)). It then queries  $H_{\text{sp}}$  with input  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell), \sigma_1)$  to find  $\hat{\mathbf{c}}$ , and returns  $\text{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{G} \cdot \hat{\mathbf{c}}, \sigma_2)$ .
- **Challenge**: Eventually  $\mathcal{A}$  sends two messages  $\text{m}_0, \text{m}_1$  to Ch, and receives back a ciphertext  $\text{ct}^* \leftarrow \text{BD.ENC}(\text{pk}, r^*, \text{m}_b)$  for  $b \leftarrow \{0, 1\}$ .
- **After some post-challenge queries**,  $\mathcal{A}$  outputs a bit  $b'$ .
- The game ends with value 1 if  $b' = b$ .

**Hybrid 1:** In hybrid 1, Ch samples  $(\mathbf{h}^*, \tilde{h}^*) \leftarrow H(r^*)$  at the setup. It then samples  $\mathbf{A}_{r^*} \leftarrow \mathbb{Z}_q^{n \times t}$  and sets the CRS as  $\text{crs}_{\text{samp}} \leftarrow \text{GenProg}(1^\lambda, 1^\ell, \tilde{h}^*, \mathbf{A}_{r^*})$ .

**Hybrid 2:** Let  $\tilde{\mathbf{R}}^* \in \{-1, 1\}^{m \times m}$  denote the random matrix sampled while creating  $\text{ct}^*$  at the challenge round. In hybrid 2, Ch chooses  $\tilde{\mathbf{R}}^*$  during the setup. It then constructs  $\hat{\mathbf{A}}$  as  $\hat{\mathbf{A}} = \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}$ .

**Hybrid 3:** To respond to  $\mathcal{A}$ 's pre- or post-challenge queries for  $\{r_i\}_{i \in [\ell]}$ , Ch first checks if  $r^* \in \{r_i\}_{i \in [\ell]}$ . If so, it aborts the game. Otherwise, it follows the same steps as in hybrid 2, except that it returns  $\text{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$  as the pre-decryption key.

**Hybrid 4:** The matrix  $\mathbf{C}$  is sampled uniformly at random,  $\mathbf{C} \leftarrow \mathbb{Z}_q^{n \times m}$ , while the matrix  $\tilde{\mathbf{B}}$  is generated along with a trapdoor  $\mathbf{T}_{\tilde{\mathbf{B}}}$ :  $(\tilde{\mathbf{B}}, \mathbf{T}_{\tilde{\mathbf{B}}}) \leftarrow \text{TrapGen}(1^n, q, t)$ .

**Hybrid 5:** Upon receiving a new query  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1)$  for the random oracle  $H_{\text{sp}}$ , Ch first parses  $\mathbf{h} = \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell)$ , and for each  $i \in [\ell]$ , checks if  $\mathbf{h}_i$  appears within the tuples  $(r_i, \mathbf{h}_i, \tilde{h}_i)$  stored after queries to  $H(\cdot)$ . In that case, given  $\mathbf{h}_i$  and  $\tilde{h}_i$ ,  $i \in [\ell]$ , it also checks if the matrices  $\mathbf{A}$ ,  $\hat{\mathbf{A}}$  and  $\mathbf{T}_{\mathbf{A}}$  are constructed as in equations (1) and (2) from  $\text{crs}_{\text{samp}}$ ,  $\tilde{\mathbf{A}}$  and  $\tilde{\mathbf{B}}$ . If these hold, Ch samples a vector  $\mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$ , where  $\mathbf{F} = [\mathbf{A} | \hat{\mathbf{A}}]$ , stores the tuple  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma, \mathbf{v})$ , and returns  $\mathbf{v}$  to  $\mathcal{A}$ . Otherwise, it responds in the same way as in hybrid 4.

**Hybrid 6:** Upon receiving a new query  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \mathbf{h}, \sigma_1)$  for the random oracle  $H_{\text{sp}}$ , Ch first parses  $\mathbf{h} = \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell)$ , and for each  $i \in [\ell]$ , checks if  $\mathbf{h}_i$  appears within the tuples  $(r_i, \mathbf{h}_i, \tilde{h}_i)$  stored after queries to  $H(\cdot)$ . In that case, given  $\mathbf{h}_i$  and  $\tilde{h}_i$ ,  $i \in [\ell]$ , it also checks if the matrices  $\mathbf{A}$ ,  $\hat{\mathbf{A}}$  and  $\mathbf{T}_\mathbf{A}$  are constructed as in equations (1) and (2) from  $\text{crs}_{\text{samp}}$ ,  $\tilde{\mathbf{A}}$  and  $\tilde{\mathbf{B}}$ . If these hold, it does the following to respond to the query:

- Ch first checks if  $\mathbf{h}^* \in \{\mathbf{h}_i\}_{i \in [\ell]}$ .
- Suppose  $\mathbf{h}^* = \mathbf{h}_{i^*}$  for some index  $i^*$ . Then, Ch samples  $\mathbf{v}_{i^*} \in \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^{2t}$  and sets  $\mathbf{c} := \mathbf{F}_{r^*} \mathbf{v}_{i^*}$ , where  $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*} | \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}]$ . It then samples  $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$  using the algorithm in Lemma 7 and sets  $\mathbf{sbk}' := \perp$ .
- For  $i \neq i^*$ ,  $i \in [\ell]$ , Ch obtains  $\mathbf{v}_i$ ,  $i \in [\ell]$ ,  $i \neq i^*$ , as follows:

$$\mathbf{v}_i \leftarrow \text{SampleRight}(\mathbf{A}_i, \tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{c} - \mathbf{h}_i, \sigma_1)$$

- Now, if  $\mathbf{h}^* \notin \{\mathbf{h}_i\}_{i \in [\ell]}$ , it samples  $\mathbf{sbk}' \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_2}^m$  and sets  $\mathbf{c} := \mathbf{C} \cdot \mathbf{sbk}'$ . It then samples  $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$  using the algorithm in Lemma 7.
- For all  $i \in [\ell]$ , Ch obtains  $\mathbf{v}_i$  as follows:

$$\mathbf{v}_i \leftarrow \text{SampleRight}(\mathbf{A}_i, \tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{c} - \mathbf{h}_i, \sigma_1)$$

- Finally, Ch returns  $\mathbf{v}$  and stores the tuple:

$$(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \mathbf{h}, \sigma, \mathbf{v} = \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2), \mathbf{sbk}')$$

Note that Ch still responds to the pre- and post-challenge queries as in hybrid 6, instead of returning  $\mathbf{sbk}'$ .

**Hybrid 7:** Given some pre- or post-challenge query  $\{r_i\}_{i \in [\ell]}$ , Ch first checks if  $r^* \in \{r_i\}_{i \in [\ell]}$ . If so, it aborts the game. Otherwise, without loss of generality, suppose  $\mathcal{A}$  has already queried  $H(\cdot)$  on the identities  $\{r_i\}_{i \in [\ell]}$ , and let  $(\mathbf{h}_i, \tilde{h}_i)$ ,  $i \in [\ell]$ , denote the responses. Then, Ch constructs the matrices  $\mathbf{A}$  and  $\hat{\mathbf{A}}$  as in equations (1) and (2). Now, without loss of generality, suppose  $\mathcal{A}$  has also already queried  $H_{\text{sp}}$  with the input  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \mathbf{h} = \text{col}(\mathbf{h}_1, \dots, \mathbf{h}_\ell), \sigma_1)$ . In that case, Ch retrieves the tuple  $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_\mathbf{A}, \mathbf{h}, \sigma, \mathbf{v} = \text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell, \hat{\mathbf{c}}, \tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_\ell), \mathbf{sbk}')$  and returns  $\mathbf{sbk}'$ .

**Completing the proof.** To complete the proof let  $\text{Hybrid}_i(\mathcal{A})$  denote the output distribution of hybrid  $i$  with adversary  $\mathcal{A}$ . We argue that the adversary's advantage changes negligibly from hybrid to hybrid, and moreover an adversary with non-negligible advantage in Hybrid 7 can be used to break the LWE assumption.

**Lemma 15.** *If  $\Pi_{\text{samp}}$  is somewhere programmable, then for all adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_0(\mathcal{A}) \approx_s \text{Hybrid}_1(\mathcal{A})$ .*

*Proof.* The only difference between these two games is the distribution of  $\text{crs}_{\text{samp}}$ , which are statistically indistinguishable by somewhere programmability of  $\Pi_{\text{samp}}$ .  $\square$

**Lemma 16.** *Suppose  $n \geq \lambda$  and  $t \geq (n + 1) \log(q) + \omega(\log(n))$ . Then, for all adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_2(\mathcal{A})$ .*



*Proof.* Recall that  $\tilde{\mathbf{R}}^*$  is used in the calculation of  $\epsilon_{3,i}^* := \text{col}(\epsilon_{3,i,\text{pre}}^*, (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,i,\text{pre}}^*)$ . Since  $t \geq (n+1)\log(q) + \omega(\log(n))$ , by Lemma 12, the tuple  $(\mathbf{A}_{r^*}, \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*, \epsilon_{3,i}^*)$  has distribution statistically close to the distribution of  $(\mathbf{A}_{r^*}, \mathbf{A}'_{r^*}, \epsilon_{3,i}^*)$ , where  $\mathbf{A}'_{r^*} \leftarrow \mathbb{Z}_q^{n \times t}$ . Therefore, the matrix  $\tilde{\mathbf{A}} := \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}$  in hybrid 2 is statistically close to uniform, *i.e.*, the matrix  $\tilde{\mathbf{A}}$  in hybrid 1. Thus, we can conclude that  $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_2(\mathcal{A})$ .  $\square$

**Lemma 17.** *Suppose  $\sigma_2 \geq m \|\mathbf{T}_C\|_\infty \cdot \omega(\sqrt{\log(n)})$ . Then, for all adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_2(\mathcal{A}) \approx_s \text{Hybrid}_3(\mathcal{A})$ .*

*Proof.* By Lemma 9, the distribution of  $\mathbf{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_C, \mathbf{c}, \sigma_2)$  in hybrid 2 is statistically indistinguishable from its distribution in hybrid 3, namely,  $\mathbf{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$ .  $\square$

**Lemma 18.** *Suppose  $n \geq \lambda$  and  $m, t \geq O(n \log(q))$ . Then, for all PPT adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_3(\mathcal{A}) \approx_s \text{Hybrid}_4(\mathcal{A})$ .*

*Proof.* In hybrid 3,  $(\mathbf{C}, \mathbf{T}_C) \leftarrow \text{TrapGen}(1^n, q, m)$  and  $\tilde{\mathbf{B}} \leftarrow \mathbb{Z}_q^{n \times t}$ , while in hybrid 4, they are sampled as  $\mathbf{C} \leftarrow \mathbb{Z}_q^{n \times m}$  and  $(\tilde{\mathbf{B}}, \mathbf{T}_{\tilde{\mathbf{B}}}) \leftarrow \text{TrapGen}(1^n, q, t)$ . Then, by Lemma 9, the distributions of  $(\tilde{\mathbf{B}}, \mathbf{C})$  are statistically indistinguishable between these two games.  $\square$

**Lemma 19.** *Suppose  $q > 2$ ,  $t > n$ , and  $\sigma_1 > \|\mathbf{T}_A\|_{\text{GS}} \cdot \omega(\log(2\ell t + m))$ . Then, for all adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_4(\mathcal{A}) \approx_s \text{Hybrid}_5(\mathcal{A})$ .*

*Proof.* By Lemma 10, output of  $\text{SampleLeft}(\mathbf{A}, \tilde{\mathbf{A}}, \mathbf{T}_A, \mathbf{h}, \sigma_1)$  is distributed statistically close to  $\mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$ , where  $\mathbf{F} = [\mathbf{A} \mid \tilde{\mathbf{A}}]$ .  $\square$

**Lemma 20.** *Suppose*

1.  $q > 2$ ,  $t, m \geq 2n \log(q)$ , and  $\sigma_1, \sigma_2 \geq \omega(\sqrt{\log(n)})$ .
2.  $\sigma_1 \geq \|\mathbf{T}_{\tilde{\mathbf{B}}}\|_{\text{GS}} \cdot s_{\mathbf{R}} \cdot \omega(\sqrt{\log(t)})$ , where  $s_{\mathbf{R}} := \sup_{\|\mathbf{x}\|=1} \|\tilde{\mathbf{R}}^* \mathbf{x}\|$ .
3. For all  $i \in [\ell]$  and  $\mathbf{h} \in \mathbb{Z}_q$ , the matrices  $\mathbf{A}_i$ ,  $\tilde{\mathbf{A}}$ ,  $\tilde{\mathbf{B}}$  and  $\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}) \tilde{\mathbf{B}}$  are full-rank (rank  $n$ ), and there exists a value  $w$  such that  $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}) \tilde{\mathbf{B}}) \geq w$ . Moreover,  $\sigma_1 \geq (q/w) \log(2\ell t)$ .
4.  $\sigma_1 \geq \|\mathbf{T}_A\|_{\text{GS}} \cdot \omega(\log(2\ell t + m))$ .

*Then, for all adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_5(\mathcal{A}) \approx_s \text{Hybrid}_6(\mathcal{A})$ .*

*Proof.* Consider the following two cases in hybrid 6:

**Case 1:**  $\mathbf{h}^* = \mathbf{h}_{i^*}$  for some  $i^*$ . As  $\tilde{\mathbf{A}} = \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}$ , it holds that

$$\mathbf{F}_{r^*} := [\mathbf{A}_{r^*} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}^*) \tilde{\mathbf{B}}] = [\mathbf{A}_{r^*} \mid \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*]$$

Here, as  $\mathbf{A}_{r^*}$  was drawn from a uniform distribution,  $\mathbf{A}_{r^*} \tilde{\mathbf{R}}^*$  is uniform in  $\mathbb{Z}_q^{n \times t}$  and independent of the other matrices by the standard leftover hash lemma. Therefore, by assumption (1) and Lemma 3,  $\mathbf{c} := \mathbf{F}_{r^*} \cdot \mathbf{v}_{i^*}$  in hybrid 6 is statistically close to a uniform sample  $\mathbf{c} \leftarrow \mathbb{Z}_q^n$ . Moreover, by Lemma 8, for uniformly-random  $\mathbf{c} \leftarrow \mathbb{Z}_q^n$ ,  $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$  is statistically close to a sample  $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$ .

**Case 2:**  $\mathbf{h}^* \notin \{\mathbf{h}_i\}_{i \in [\ell]}$ . Again by assumption (1) and Lemma 3,  $\mathbf{c} := \mathbf{C} \cdot \mathbf{sbk}'$  in hybrid 6 is statistically close to a uniform sample  $\mathbf{c} \leftarrow \mathbb{Z}_q^n$ . Again by Lemma 8,  $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$  is statistically close to a sample  $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$ .



In case (2), for all  $i \in [\ell]$ , it holds that

$$\mathbf{F}_i := [\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_i)\tilde{\mathbf{B}}] = [\mathbf{A}_i \mid \mathbf{A}_{r^*}\tilde{\mathbf{R}}^* + (H_{\text{frd}}(\mathbf{h}_i) - H_{\text{frd}}(\mathbf{h}^*))\tilde{\mathbf{B}}],$$

where  $H_{\text{frd}}(\mathbf{h}_i) - H_{\text{frd}}(\mathbf{h}^*) \in \mathbb{Z}_q^{n \times n}$  is full-rank (rank  $n$ ) as  $\mathbf{h}_i \neq \mathbf{h}^*$  for any  $i \in [\ell]$  with high probability. Thus,  $(H_{\text{frd}}(\mathbf{h}_i) - H_{\text{frd}}(\mathbf{h}^*))\tilde{\mathbf{B}}$  is full-rank, and  $\mathbf{T}_{\tilde{\mathbf{B}}}$  is a trapdoor for that matrix. Then, by assumption (2) and Lemma 11, the output  $\mathbf{v}_i$  of

$$\text{SampleRight}(\mathbf{A}_i, \tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{c} - \mathbf{h}_i, \sigma_1)$$

in hybrid 6 is statistically close to a sample from  $(\mathbf{F}_i)_{\sigma_1}^{-1}(\mathbf{c} - \mathbf{h}_i)$ .

By the same reasoning as above,  $\mathbf{v}_i$ ,  $i \in [\ell]$ ,  $i \neq i^*$ , are also statistically close to samples from  $(\mathbf{F}_i)_{\sigma_1}^{-1}(\mathbf{c} - \mathbf{h}_i)$  in case (1). By definition of  $\mathbf{c}$  in case (1),  $\mathbf{v}_{i^*}$  is statistically close to a sample from the distribution  $(\mathbf{F}_{r^*})_{\sigma_1}^{-1}(\mathbf{c} - \mathbf{h}^*)$ , i.e.,  $(\mathbf{F}_{i^*})_{\sigma_1}^{-1}(\mathbf{c} - \mathbf{h}_{i^*})$  in case (1).

Finally, by assumption (3) and Lemma 4, the statistical distance between the following distributions is negligible for  $F = [\mathbf{A} \mid \hat{\mathbf{A}}]$ :

$$\left\{ \mathbf{v} : \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{u}) \right\} \quad \text{and} \quad \left\{ \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) : \begin{array}{l} \hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^k \\ \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow ([\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h}_i)\tilde{\mathbf{B}}]_{\sigma_1}^{-1}(\mathbf{h}_i - \mathbf{G}\hat{\mathbf{c}})) \end{array} \right\}$$

By assumption (4), this shows that the distribution of  $v$  in hybrid 6 is statistically close to its distribution in hybrid 5.  $\square$

**Lemma 21.** *For all PPT adversaries  $\mathcal{A}$ ,  $\text{Hybrid}_6(\mathcal{A}) \equiv \text{Hybrid}_7(\mathcal{A})$ .*

*Proof.* Since  $\mathbf{sbk} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_2}^m$  and  $\mathbf{c} := \mathbf{C} \cdot \mathbf{sbk}$  in hybrid 6, distribution of the pre-decryption key in hybrid 7 is indistinguishable from its distribution in hybrid 6.  $\square$

**Lemma 22.** *Suppose  $n \geq \lambda$ ,  $q > 2$ ,  $m \geq 3n \lceil \log q \rceil$ ,  $t = m(\beta \lceil \log \ell \rceil + 1)$ ,  $\sigma_1 \geq (\ell t + m) \cdot \omega(\log(2\ell t + m))$  and  $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$ . Then,*

1. *Construction 4.6 by Waters et al. [46] (cf. Lemma 14) is an  $(n, t, q, \sigma_1)$ -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion.*
2.  $\sigma_1 \geq \|\mathbf{T}_{\mathbf{A}}\|_{\text{GS}} \cdot \omega(\log(2\ell t + m))$ .
3.  $\sigma_1 \geq \|\mathbf{T}_{\tilde{\mathbf{B}}}\|_{\text{GS}} \cdot s_{\tilde{\mathbf{R}}^*} \cdot \omega(\sqrt{\log(t)}) \geq \omega(\sqrt{\log(n)})$ .
4.  $\sigma_2 \geq m \|\mathbf{T}_{\mathbf{C}}\|_{\infty} \cdot \omega(\sqrt{\log(n)}) \geq \omega(\sqrt{\log(n)})$ .
5.  $t, m \geq O(n \log(q))$  and  $t \geq (n + 1) \log(q) + \omega(\log(n))$ .
6. *For all  $i \in [\ell]$  and  $\mathbf{h} \in \mathbb{Z}_q$ , the matrices  $\mathbf{A}_i$ ,  $\tilde{\mathbf{A}}$ ,  $\tilde{\mathbf{B}}$  and  $\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h})\tilde{\mathbf{B}}$  are full-rank (rank  $n$ ), and there exists a value  $w$  such that  $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h})\tilde{\mathbf{B}}) \geq w$ . Moreover,  $\sigma_1 \geq (q/w) \log(2\ell t)$ .*

Here,  $s_{\mathbf{R}} := \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$ .

*Proof.* Since  $n \geq \lambda$ ,  $m = 3n \lceil \log q \rceil$ ,  $t = m(\beta \lceil \log \ell \rceil + 1)$  and  $\sigma_1 \geq (\ell t + m) \cdot \omega(\log(2\ell t + m)) \geq (\ell t + m) \log(\ell n)$ , by Lemma 14, construction 4.6 by Waters et al. [46] (cf. Lemma 14) is an  $(n, t, q, \sigma_1)$ -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion (item (1)). Therefore,  $\|\mathbf{T}_{\mathbf{A}}\|_{\infty} \leq 1$  and  $\|\mathbf{T}_{\mathbf{A}}\|_{\text{GS}} \leq \|\mathbf{T}_{\mathbf{A}}\| \leq \sqrt{\ell t + m} \|\mathbf{T}_{\mathbf{A}}\|_{\infty} \leq \sqrt{\ell t + m}$ . This implies  $\|\mathbf{T}_{\mathbf{A}}\|_{\text{GS}} \cdot \omega(\log(2\ell t + m)) \leq \sqrt{\ell t + m} \cdot \omega(\log(2\ell t + m)) \leq \sigma_1$  (item 2).

Next, by Lemma 9,  $\|\mathbf{T}_{\tilde{\mathbf{B}}}\|_{\infty} \leq 1$ , which implies  $\|\mathbf{T}_{\tilde{\mathbf{B}}}\|_{\text{GS}} \leq \|\mathbf{T}_{\tilde{\mathbf{B}}}\| \leq \sqrt{t}$ . Moreover, by Lemma 1 and the fact that  $t \geq O(n \log(q))$ ,  $s_{\tilde{\mathbf{R}}^*} \leq O(\sqrt{t})$ . Consequently,  $\|\mathbf{T}_{\tilde{\mathbf{B}}}\|_{\text{GS}} \cdot s_{\tilde{\mathbf{R}}^*} \cdot \omega(\sqrt{\log(t)}) \leq O(t) \cdot \omega(\sqrt{\log(t)}) \leq \sigma_1$  (item 3).

Since  $\|\mathbf{T}_{\mathbf{C}}\|_{\infty} \leq 1$  by Lemma 9,  $\omega(\sqrt{\log(n)}) \leq m\|\mathbf{T}_{\mathbf{C}}\|_{\infty} \cdot \omega(\sqrt{\log(n)}) \leq m \cdot \omega(\sqrt{\log(n)}) \leq \sigma_2$  (item 4).

Item 5 follows from the assumption on the sizes of  $m$  and  $t$ .

Finally, by Lemmas 6 and 9, for all  $i \in [\ell]$  and  $\mathbf{h} \in \mathbb{Z}_q^n$ ,  $\lambda_1^{\infty}(\mathbf{A}_i), \lambda_1^{\infty}(\tilde{\mathbf{A}} + H_{\text{frd}}(\mathbf{h})\tilde{\mathbf{B}}) \geq w = q/4$ , which implies that  $(q/w) \log(2\ell t) \leq 4 \log(2\ell t) \leq \sigma_1$ . Furthermore, by Lemmas 9 and 14, these matrices are full-rank (rank  $n$ ). This concludes item (6).  $\square$

Lemma 22 implies that all previous assumptions hold, *i.e.*,  $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_7(\mathcal{A})$ . Hence, to conclude the proof of Theorem 2, we construct an adversary  $\mathcal{B}$  that solves the  $(\mathbb{Z}_q, n, \Psi_{\alpha})$ -LWE problem by acting as a challenger of hybrid 7 for  $\mathcal{A}$ . Upon receiving the LWE challenges  $\{(\mathbf{y}_i, z_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q\}_{i \in [t+m+1]}$ , at the setup,  $\mathcal{B}$  sets

$$\mathbf{h}^* := \mathbf{y}_0 \quad \mathbf{C} := [\mathbf{y}_1, \dots, \mathbf{y}_m] \quad \mathbf{A}_{r^*} := [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]$$

It constructs the rest of the public key as in hybrid 7, and answers the pre- and post-challenge queries as in hybrid 7.

For the challenge ciphertext,  $\mathcal{B}$  sets

$$\begin{aligned} \text{ct}^*[1] &:= z_0 + \mathbf{m}_b \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}^*[2] &:= \text{col}(z_1, \dots, z_m) \in \mathbb{Z}_q^m \\ \text{ct}^*[3] &:= \begin{bmatrix} \text{col}(z_{m+1}, \dots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^{\top} \text{col}(z_{m+1}, \dots, z_{m+t}) \end{bmatrix} \in \mathbb{Z}_q^{2t} \end{aligned}$$

We note that when the LWE oracle is pseudorandom (*i.e.*,  $\mathcal{O} = \mathcal{O}_s$ ), then  $\text{ct}^*$  has the same distribution as in hybrid 7. This is because  $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*} | \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*]$  in hybrid 7, and by the definition of  $\mathcal{O}_s$ ,

$$\begin{aligned} \text{ct}^*[1] &= z_0 + \mathbf{m}_b \lfloor q/2 \rfloor = (\mathbf{h}^*)^{\top} \mathbf{s} + \epsilon_1^* \\ \text{ct}^*[2] &= \text{col}(z_1, \dots, z_m) = [\mathbf{y}_1, \dots, \mathbf{y}_m]^{\top} \mathbf{s} + \epsilon_2^* = \mathbf{C}^{\top} \mathbf{s} + \epsilon_2^* \\ \text{ct}^*[3] &= \begin{pmatrix} \text{col}(z_{m+1}, \dots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^{\top} \text{col}(z_{m+1}, \dots, z_{m+t}) \end{pmatrix} = \begin{pmatrix} [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]^{\top} \mathbf{s} + \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^{\top} [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]^{\top} \mathbf{s} + (\tilde{\mathbf{R}}^*)^{\top} \epsilon_{3,\text{pre}}^* \end{pmatrix} \\ &= \begin{pmatrix} \mathbf{A}_{r^*}^{\top} \mathbf{s} + \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^{\top} \mathbf{A}_{r^*}^{\top} \mathbf{s} + (\tilde{\mathbf{R}}^*)^{\top} \epsilon_{3,\text{pre}}^* \end{pmatrix} = \mathbf{F}_{\mathbf{h}^*}^{\top} \mathbf{s} + \begin{pmatrix} \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^{\top} \epsilon_{3,\text{pre}}^* \end{pmatrix} = \mathbf{F}_{\mathbf{h}^*}^{\top} \mathbf{s} + \epsilon_3^* \end{aligned}$$

for some short noise vectors  $\epsilon_1^* \leftarrow \Psi_{\alpha}$ ,  $\epsilon_2^* \leftarrow \Psi_{\alpha}^m$  and  $\epsilon_{3,\text{pre}}^* \leftarrow \Psi_{\alpha}^t$ .

Now, when  $\mathcal{O} = \mathcal{O}_{\S}$ ,  $\text{ct}^*[1]$  and  $\text{ct}^*[2]$  are uniformly-random samples from  $\mathbb{Z}_q$  and  $\mathbb{Z}_q^m$  respectively. Similarly,  $\text{col}(z_{m+1}, \dots, z_{m+t})$  is a random sample from  $\mathbb{Z}_q^t$ , and  $(\tilde{\mathbf{R}}^*)^{\top} \text{col}(z_{m+1}, \dots, z_{m+t})$  is uniform in  $\mathbb{Z}_q^t$  and independent of the other vectors by the standard leftover hash lemma. Here, the hash function is defined by the matrix  $[\mathbf{A}_{r^*}^{\top} | \text{col}(z_{m+1}, \dots, z_{m+t})]$  and ensures that  $\mathbf{A}_{r^*} \tilde{\mathbf{R}}^*$  and  $(\tilde{\mathbf{R}}^*)^{\top} \text{col}(z_{m+1}, \dots, z_{m+t})$  are uniformly distributed and independent quantities. Hence, when  $\mathcal{O} = \mathcal{O}_{\S}$ , the challenge ciphertext is always uniform in  $\mathbb{Z}_q \times \mathbb{Z}_q^m \times \mathbb{Z}_q^{2t}$ .

Finally, we observe that if  $\mathcal{A}$  has advantage  $\epsilon$ , then  $\mathcal{B}$  has advantage at least  $\epsilon$  in deciding the  $(\mathbb{Z}_q, n, \Psi_{\alpha})$ -LWE problem. This concludes the proof.  $\square$

### 4.3 Multi-bit Encryption and Adaptive Security

To encrypt messages with  $N$  bits, as in [31], the hash function  $H(\cdot)$  can be made to output  $N$  vectors in  $\mathbb{Z}_q^n$ , each used to encrypt one of the message bits with the same ephemeral  $s \in \mathbb{Z}_q^n$ . This increases the size of  $\text{ct}_r[1]$  by a factor of  $N$ , while keeping  $\text{ct}_r[2]$  and  $\text{ct}_r[3]$  the same. The security proof remains largely unchanged, except that the challenger must now query the LWE oracle  $N + m + t$  times instead of  $1 + m + t$  times. An alternative method would be representing the messages as integers in the range  $[0, 2^\ell]$ , and calculating  $\text{ct}_r[1]$  as  $\text{ct}_r[1] := \mathbf{h}_r^\top \mathbf{s}_r + \epsilon_{1,r} + \mathbf{m} \cdot \lfloor q/2^N \rfloor$ . This in turn requires the error term to be less than  $q/(1 + 2^{N+1})$ , which increases  $q$  by a factor of  $2^N$ . Whereas the first solution increases the ciphertext size by  $O(N(\log(\ell) + \log(\lambda)))$  bits, the latter results in a factor of  $N$  increase.

For adaptive security, one can use a technique called complexity leveraging to move from static to adaptive security. The approach is based on guessing the adaptively chosen  $r^*$  at the beginning of the game. More specifically, suppose the adversary queries the random oracle  $H(\cdot)$   $Q_{\text{hash}}$  times in total. The challenger can select a uniformly random index  $i \leftarrow Q_{\text{hash}}$ , as well as uniformly random vectors  $\mathbf{h}^* \leftarrow \mathbb{Z}_q^n$  and  $\tilde{h}^* \leftarrow \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$  in advance, and return  $(\mathbf{h}^*, \tilde{h}^*)$  upon receiving the  $i$ -th query, denoted by  $r^*$ , to  $H(\cdot)$ . It aborts if the adversary does not specify  $r^*$  as its challenge index, or asks for the pre-decryption key for a batch of indices containing  $r^*$ . Note that the probability  $\text{Ch}$  does not abort during this simulation is larger than  $1/Q_{\text{hash}}$  times the probability it does not abort in the selective security game. Moreover, conditioned on the fact that  $\text{Ch}$  does not abort, the view it provides to  $\mathcal{A}$  is statistically close to the view of the selective security game. This concludes the proof of adaptive security with a factor of  $Q_{\text{hash}}$  security loss, which is polynomial in the security parameter  $\lambda$ .

### 4.4 Reducing the Size of Public Key

Recall that the public key and ciphertexts suffer from a  $\beta$  factor increase in size. In this section, we propose a simple solution to keep  $\beta = 1$  and the pre-decryption key size small, while avoiding collisions in the same identity.

Suppose each encryptor encrypts its message under  $2 \lceil \log(\ell) \rceil$  distinct identities using  $\beta = 1$ . Then, upon receiving ciphertexts from  $\ell/2$  encryptors, the pre-decryption algorithm tries to assign a unique bit vector  $\tilde{h}_i$  to each encryptor from among the  $2 \lceil \log(\ell) \rceil$  bit vectors corresponding to the encryptor's  $2 \lceil \log(\ell) \rceil$  identities. This is a classical problem of balls and bins, and when all of the  $\ell/2$  encryptors propose  $2 \lceil \log(\ell) \rceil$  choices, each can be assigned to a unique index with high probability, using an online algorithm (*cf.* [4, Theorem 4]). Therefore, the pre-decryption algorithm succeeds in its assignment with high probability, and pads the remaining  $\ell/2$  positions in the batch with random identities and the remaining  $\ell/2$  distinct bit vectors. This solution results in a  $2 \log(\ell)$  factor increase in the size of ciphertexts, as well as an effective  $2 \times$  increase in the size of pre-decryption keys.

*Remark 4.* When our batch decryption scheme is used for encrypted pools, it is not necessary for all of the  $2 \lceil \log \ell \rceil$  ciphertexts to be observed by all decryptors (*i.e.*, all blockchain nodes). Only the block proposer must receive these ciphertexts to select a unique bit vector  $\tilde{h}_i$  for each message (transaction). After one of the  $2 \lceil \log \ell \rceil$  ciphertexts is placed in the block for each message, the rest of the ciphertexts can be discarded. To decrypt the ciphertexts within a block, the other blockchain nodes do not need any more data than the pre-decryption key. This implies an effective  $\log(\ell)$ -factor reduction in the size of ciphertexts downloaded by the blockchain nodes other than the proposer.

## Acknowledgements

We thank Gregory Neven for helpful discussions about this project. This work was partially supported by the Simons Foundation, the Stanford Center for Blockchain Research, and UBRI. The third author was further supported by a Research Hub Collaboration agreement between Stanford University and Input Output Global Inc.

## References

1. Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. 2024. To appear in CRYPTO 2025.
2. Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, volume 6110 of *Lecture Notes in Computer Science*, pages 553–572. Springer, 2010.
3. Martin R. Albrecht, Valerio Cini, Russell W. F. Lai, Giulio Malavolta, and Sri Aravinda Krishnan Thyagarajan. Lattice-based snarks: Publicly verifiable, preprocessing, and recursively composable - (extended abstract). In *CRYPTO (2)*, volume 13508 of *Lecture Notes in Computer Science*, pages 102–132. Springer, 2022.
4. Noga Alon, Eyal Lubetzky, and Ori Gurel-Gurevich. Choice-memory tradeoff in allocations. In *FOCS*, pages 230–238. IEEE Computer Society, 2009.
5. Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022.
6. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In *TCC*, volume 5978 of *Lecture Notes in Computer Science*, pages 201–218. Springer, 2010.
7. Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *Lecture Notes in Computer Science*, pages 218–236. Springer, 2013.
8. Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer, 2004.
9. Dan Boneh and Xavier Boyen. Efficient selective identity-based encryption without random oracles. *J. Cryptol.*, 24(4):659–693, 2011.
10. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In *CT-RSA*, volume 3860 of *Lecture Notes in Computer Science*, pages 226–243. Springer, 2006.
11. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. Cryptology ePrint Archive, Paper 2025/279, 2025.
12. Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. *SIAM J. Comput.*, 36(5):1301–1328, 2007.
13. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer, 2001.
14. Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In *EUROCRYPT*, volume 8441 of *Lecture Notes in Computer Science*, pages 533–556. Springer, 2014.
15. Dan Boneh, Craig Gentry, and Michael Hamburg. Space-efficient identity based encryption without pairings. In *FOCS*, pages 647–657. IEEE Computer Society, 2007.
16. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *CRYPTO (1)*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428. Springer, 2013.
17. Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2008.
18. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. BEAT-MEV: epochless approach to batched threshold encryption for MEV prevention. *IACR Cryptol. ePrint Arch.*, page 1533, 2024. To appear in Usenix Security 2025.
19. David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai trees, or how to delegate a lattice basis. *J. Cryptol.*, 25(4):601–639, 2012.
20. Dario Catalano and Dario Fiore. Vector commitments and their applications. In *Public Key Cryptography*, volume 7778 of *Lecture Notes in Computer Science*, pages 55–72. Springer, 2013.
21. Dario Catalano, Dario Fiore, and Mariagrazia Messina. Zero-knowledge sets with short proofs. In *EUROCRYPT*, volume 4965 of *Lecture Notes in Computer Science*, pages 433–450. Springer, 2008.

22. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *USENIX Security Symposium*. USENIX Association, 2024.
23. Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. *IACR Cryptol. ePrint Arch.*, page 1516, 2024. To appear in Usenix Security 2025.
24. Valerio Cini and Hoeteck Wee. ABE for circuits with poly( $\lambda$ )-sized keys from LWE. In *FOCS*, pages 435–446. IEEE, 2023.
25. Clifford C. Cocks. An identity based encryption scheme based on quadratic residues. In *IMACC*, volume 2260 of *Lecture Notes in Computer Science*, pages 360–363. Springer, 2001.
26. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *SP*, pages 910–927. IEEE, 2020.
27. Yevgeniy Dodis, Leonid Reyzin, and Adam D. Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In *EUROCRYPT*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540. Springer, 2004.
28. Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wöhrig. Mcfly: Verifiable encryption to the future made practical. In *FC (1)*, volume 13950 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2023.
29. Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In *EUROCRYPT (2)*, volume 14652 of *Lecture Notes in Computer Science*, pages 311–341. Springer, 2024.
30. Stefan Dziembowski, Sebastian Faust, and Jannik Luh. Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981, 2024.
31. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC*, pages 197–206. ACM, 2008.
32. Rishab Goyal and Vinod Vaikuntanathan. Locally verifiable signature and key aggregation. In *CRYPTO (2)*, volume 13508 of *Lecture Notes in Computer Science*, pages 761–791. Springer, 2022.
33. Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *CCS*, pages 89–98. ACM, 2006.
34. Benoît Libert and Moti Yung. Concise mercurial vector commitments and independent zero-knowledge sets with short proofs. In *TCC*, volume 5978 of *Lecture Notes in Computer Science*, pages 499–517. Springer, 2010.
35. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, 2012.
36. Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. Fairblock: Preventing blockchain front-running with minimal overheads. In *SecureComm*, volume 462 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pages 250–271. Springer, 2022.
37. Chris Peikert, Zachary Pepin, and Chad Sharp. Vector and functional commitments from lattices. In *TCC (3)*, volume 13044 of *Lecture Notes in Computer Science*, pages 480–511. Springer, 2021.
38. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, pages 84–93. ACM, 2005.
39. Michael Reiter and Ken Birman. How to securely replicate services. *ACM TOPLAS*, 16(3):986–1009, 1994.
40. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *EUROCRYPT*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473. Springer, 2005.
41. Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.
42. Adi Shamir. Identity-based cryptosystems and signature schemes. In *CRYPTO*, volume 196 of *Lecture Notes in Computer Science*, pages 47–53. Springer, 1984.
43. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997.
44. Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. *IACR Cryptol. ePrint Arch.*, page 762, 2024.
45. Hong Wang, Yuqing Zhang, and Dengguo Feng. Short threshold signature schemes without random oracles. In *INDOCRYPT 2005*, volume 3797 of *Lecture Notes in Computer Science*, pages 297–310. Springer, 2005.
46. Brent Waters, Hoeteck Wee, and David J. Wu. New techniques for preimage sampling: Improved nizks and more from LWE. *IACR Cryptol. ePrint Arch.*, page 1401, 2024. Eurocrypt 2025.
47. Hoeteck Wee. Circuit ABE with poly(depth,  $\lambda$ )-sized ciphertexts and keys from lattices. *IACR Cryptol. ePrint Arch.*, page 1416, 2024.
48. Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In *EUROCRYPT (3)*, volume 14006 of *Lecture Notes in Computer Science*, pages 385–416. Springer, 2023.

49. Mark Zhandry. To label, or not to label (in generic groups). In *CRYPTO (3)*, volume 13509 of *Lecture Notes in Computer Science*, pages 66–96. Springer, 2022.
50. Richard Zippel. Probabilistic algorithms for sparse polynomials. In *EUROSAM*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.

## A Extended Preliminaries

### A.1 Batch Threshold Decryption

**Definition 8.** A batch threshold decryption (BTD) scheme instantiated with a committee of  $N \in \mathbb{N}$  parties, a threshold  $t \in [N]$ , a batch size of  $\ell \in \mathbb{N}$ , a message space  $\mathcal{M}$  and an identity space  $\mathcal{I}$  consists of four PPT algorithms:

- $\text{BTD.SETUP}(1^\lambda, 1^\ell, 1^N, 1^t) \rightarrow (\text{pk}, \text{sk}_1, \dots, \text{sk}_N)$ : The setup algorithm takes as input the security parameter  $\lambda$ , the batch size  $\ell$ , the number of parties  $N$  and the threshold  $t$ , and outputs a public key  $\text{pk}$  and  $N$  secret keys  $(\text{sk}_1, \dots, \text{sk}_N)$ .
- $\text{BTD.ENC}(\text{pk}, r, m) \rightarrow \text{ct}$ : The encryption algorithm takes as input a public key  $\text{pk}$ , an identity  $r \in \mathcal{I}$  and a message  $m \in \mathcal{M}$ , and outputs a ciphertext  $\text{ct}$ .
- $\text{BTD.PREDEC}(\text{sk}_i, \{r_i\}_{i \in [\ell]}) \rightarrow \text{sbk}_i$ : The predecryption algorithm takes as input a secret key  $\text{sk}_i$  for a party  $i \in [N]$  and  $\ell$  identities, and outputs a predecryption key  $\text{sbk}_i$  for the party  $i$ .
- $\text{BTD.COMBINE}(\text{pk}, T \subseteq [N], \{r_i\}_{i \in T}, \{\text{sbk}_i\}_{i \in T}) \rightarrow \text{sbk}$ : The combiner algorithm takes as input a subset  $T \subseteq [N]$ ,  $\ell$  identities, and the predecryption keys  $\{\text{sbk}_i\}_{i \in T}$  of  $|T| = t$  parties, and outputs a single short predecryption key  $\text{sbk}$ .
- $\text{BTD.DEC}(\text{pk}, \text{sbk}, \{\text{ct}_i\}_{i \in [\ell]}) \rightarrow \{m_i\}_{i \in [\ell]}$ : The decrypt algorithm takes as input a combined predecryption key  $\text{sbk}$  and  $\ell$  ciphertext, and outputs the corresponding messages.

**Definition 9 (Correctness of BTD).** For all  $\lambda, \ell \in \mathbb{N}$ ,  $T \subseteq [N]$ ,  $\{m_i\}_{i \in [\ell]} \in \mathcal{M}^\ell$  and  $\{r_i\}_{i \in [\ell]} \in \mathcal{I}^\ell$ , the following probability is greater than  $1 - \text{negl}(\lambda)$ :

$$\Pr \left[ \text{BTD.DEC}(\text{pk}, \text{sbk}, \{\text{ct}_i\}_{i \in [\ell]}) = \{m_i\}_{i \in [\ell]} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell, 1^N, 1^t), \\ \text{ct}_i \leftarrow \text{BTD.ENC}(\text{pk}, r_i, m_i), \\ \text{sbk}_i \leftarrow \text{BTD.PREDEC}(\text{sk}_i, \{r_i\}_{i \in [\ell]}) \quad \forall i \in [\ell] \\ \text{sbk} \leftarrow \text{BTD.COMBINE}(\text{pk}, T, \{r_i\}_{i \in T}, \{\text{sbk}_i\}_{i \in T}) \end{array} \right]$$

**Definition 10 (Security of BTD).** The security game  $\text{Game}_{\mathcal{A}, b}^{\text{BTD}}(1^\lambda, \ell, t, N)$  is defined below with respect to the adversary  $\mathcal{A}$ . A BTD scheme is selectively secure in the random oracle model, if for all  $\ell, N = \text{poly}(\lambda)$ ,  $t < N$  and PPT adversaries  $\mathcal{A}$ , there exists some negligible function  $\text{negl}(\cdot)$  such that

$$|\Pr[\text{Game}_{\mathcal{A}, 0}^{\text{BTD}}(1^\lambda, \ell, t, N) = 1] - \Pr[\text{Game}_{\mathcal{A}, 1}^{\text{BTD}}(1^\lambda, \ell, t, N) = 1]| < \text{negl}(\lambda)$$

#### The Security Game $\text{Game}_{\mathcal{A}, b}^{\text{BTD}}(1^\lambda, \ell, t, N)$

**Challenge identity:** The adversary commits to the challenge identity  $r^*$  and sends it to the challenger.

**Setup:** The adversary sends polynomially bounded parameters  $N, t \leq N$  and a subset  $S \subseteq [N]$  of size  $t - 1$  to the challenger. The challenger also takes as input the security parameter  $\lambda$  and the batch size  $\ell$ , and runs  $(\text{pk}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{BTD.SETUP}(1^\lambda, 1^\ell, 1^N, 1^t)$ . It sends  $\text{pk}$  and  $\{\text{sk}_i\}_{i \in S}$  to  $\mathcal{A}$ .

**Pre-challenge queries:** The adversary may issue an arbitrary number of pre-challenge queries for computing the predecryption key:

- $\mathcal{A}$  sends a list  $\{r_i\}_{i \in [\ell]}$  of up to  $\ell$  identities to Ch.
- Ch computes all  $N$  predecryption key shares for  $i \in [N]$ ,  
 $\text{sbk}_i \leftarrow \text{BD.PREDEC}(\text{sk}_i, \{r_i\}_{i \in [\ell]}),$  and sends  $\{\text{sbk}_i\}_{i \in [N]}$  to  $\mathcal{A}$ .

**Challenge round:** At any point in time, the adversary may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$  sends two messages  $m_0, m_1 \in \mathcal{M}$  on which, it would like to be challenged.
- If the adversary has previously asked for the predecryption key shares corresponding to a list of identities  $\{r_i\}_{i \in [\ell]}$  such that  $r^* \in \{r_i\}_{i \in [\ell]}$ , then Ch halts the game.
- Otherwise, Ch computes  $\text{ct}_b \leftarrow \text{BD.ENC}(\text{pk}, r^*, m_b)$  for  $b \leftarrow \{0, 1\}$  and sends  $\text{ct}_b$  to  $\mathcal{A}$ .

**Post-challenge queries:** The adversary may again issue an arbitrary number of key computation queries as in the pre-challenge case.

**Random oracle queries:** Upon receiving a query containing a new input  $x$ , Ch samples a value  $y$  from the output distribution of the random oracle, stores the tuple  $(x, y)$ , and returns  $y$ . It returns the value  $y$  upon receiving the same input  $x$  in a future query.

**Output:** At any point in time,  $\mathcal{A}$  can decide to output a bit  $b' \in \{0, 1\}$ . The game then halts with the same output  $b'$ .

## A.2 BEAT-MEV: Batch (Threshold) Decryption by Bormet et al. [18].

Bormet et al. introduced the first efficient batch threshold decryption scheme without epochs [18]. At the core of BEAT-MEV is a *key-homomorphic puncturable PRF*, adapted from the PRF by Dujmovic et al. [29]. Each encryptor evaluates the PRF at a unique identity  $i$  using a PRF key  $k_i$  and masks its message by the PRF output:  $\gamma_i = m_i + \text{PRF}(k_i, i)$ . The masked messages are published together with an ElGamal encryption of the key  $k_i$  (in the exponent) and the punctured key  $k_i^*$  that is punctured at the identity  $i$ . The secret key  $K$ , created for a batch of  $\ell$  identities, is computed using the homomorphic property of ElGamal encryption and satisfies  $\text{PRF}(K, i) = \sum_{j \in [\ell]} \text{PRF}(k_j, i)$ . Finally, decryption recovers  $m_i$  by calculating

$$\gamma_i - \text{PRF}(K, i) + \sum_{j \in [\ell], j \neq i} \text{PuncturedPRF}(k_j^*, i),$$

since  $\text{PRF}(K, i) = \sum_{j \in [\ell]} \text{PRF}(k_j, i)$ , and  $\text{PuncturedPRF}(k_j^*, i) = \text{PRF}(k_j, i)$  for  $j \neq i$ .

The predecryption key is a single BLS12-381  $\mathbb{G}_2$  group element, and each ciphertext contains 3 group elements from  $\mathbb{G}_1$  (i.e., the punctured key and the ElGamal ciphertext) and one group element from  $\mathbb{G}_T$  (i.e.,  $\gamma$ ). However, the public keys for the key-homomorphic puncturable PRF have size quadratic in the domain of the PRF. Although the domain can be restricted to  $\ell$  integers, as the success of decryption requires the PRF keys to be punctured at unique integers, this would require advance coordination among the encryptors. To remove coordination, the authors propose splitting a batch into multiple sub-batches, each containing ciphertexts with distinct identities, and publishing secret keys for each sub-batch (cf., [18, §5.3] for numerical analysis). Since the decryption algorithm requires  $O(\ell^2)$  pairings in total, this technique also reduces the computational burden of decryption, however at the expense of larger pre-decryption keys and ciphertexts.

**Lattice-based BEAT-MEV.** We briefly describe the version of BEAT-MEV that uses the puncturable key-homomorphic PRF construction of Boneh, Lewi, Montgomery, and Raghunathan (BLMR) [16, §5] and estimate its parameter sizes.

The public parameters of the BLMR PRF consist only of two random matrices  $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{m \times m}$ , and its key is a vector  $\mathbf{sk} \in \mathbb{Z}_q^m$ . The PRF is evaluated at a point  $x = x_1 \dots, x_\ell \in \{0, 1\}^k$  as follows:

$$\text{PRF}(\mathbf{sk}, x) = \left\lfloor \prod_{i=1}^k \mathbf{A}_{x_i} \cdot \mathbf{sk} \right\rfloor_p,$$

This ensures that  $\text{PRF}(\mathbf{sk}_1 + \mathbf{sk}_2, x) = \text{PRF}(\mathbf{sk}_1, x) + \text{PRF}(\mathbf{sk}_2, x) + e$ , where the error term is  $e \in \{0, 1, 2\}^m$ . Here, the security parameter is  $n$ , and  $q = O(\sqrt{n}2^{n^\epsilon})$ , where  $\epsilon \in [0, 1]$  links the LWE problem to the problem of approximating the worst-case GapSVP to  $\tilde{O}(n/2^{n^\epsilon})$  factors. Moreover,  $m = \lceil n \log q \rceil$ ,  $k = n^\epsilon / \log(n)$  and  $p = 2^{n^\epsilon - \omega(\log(n))}$ , which implies  $m = O(kn \log(n))$  and  $\log(p) = O(k \log(n))$ .

One can view the evaluation of the BLMR PRF as calculating a single branch in a tree of evaluations, whose leaf at an index  $x$  is the PRF evaluated at index  $x$ . Let  $u_{\perp, x_1, \dots, x_{k'}}$ ,  $k' \leq k$ , denote the inner node at depth  $k'$  of the tree that is reached from the root by moving to the left child at depth  $i$ , if  $x_i = 0$ , and to the right child, if  $x_i = 1$ , proceeding through the depths  $i = 1, \dots, k$ . Then,

$$u_{\perp, x_1, \dots, x_{k'}} = \left\lfloor \prod_{i=1}^{k'} \mathbf{A}_{x_i} \cdot \mathbf{sk} \right\rfloor_p,$$

Now, the BLMR PRF can be punctured at a point  $x$  by publishing as the punctured key, all the inner nodes that correspond to the siblings of the nodes on the path from the leaf  $x$  to the root. In other words, the punctured key consists of  $u_{\perp, \bar{x}_1}, u_{\perp, x_1, \bar{x}_2}, \dots, u_{\perp, x_1, \dots, x_{k-1}, \bar{x}_k}$ . This enables calculating the PRF at any value  $x' \neq x$  that first differs from  $x$  at some index  $j$  (i.e.,  $x'_i = x_i$  for  $i < j$ , and  $x'_j \neq x_j$ ) as follows:

$$\text{PRF}(\mathbf{sk}, x) = \left\lfloor u_{\perp, x_1, \dots, x_{j-1}, \bar{x}_j} \prod_{i=j+1}^k \mathbf{A}_{x_i} \right\rfloor_{p'},$$

where  $p' = O(m^k p)$  is made larger to account for the increase in the noise when multiplying  $\prod_{i=j+1}^k \mathbf{A}_{x_i}$  with the error term  $e$  associated with the inner node. This implies  $\log(p') = k \log(m) + \log(p) = O(k(\log(k) + \log(n)))$ .

Since BEAT-MEV requires the encryptors to publish their encrypted PRF keys through a homomorphic encryption scheme, we must also pick an appropriate homomorphic scheme for encrypting the BLMR PRF keys. We choose Regev encryption to preserve the structure of the PRF keys [38]. This also enables thresholdizing the new BEAT-MEV scheme by using a threshold version of Regev encryption [6].

Finally, we express the parameter sizes of the batch decryption scheme in terms of our notation for the security parameter, namely  $\lambda$ . As the matrices  $\mathbf{A}_0$  and  $\mathbf{A}_1$  are random, they can be generated from a small seed of size  $O_\lambda(1)$  using a pseudo-random generator. Since Regev encryption has public keys of size  $O_\lambda(1)$ , the public key of the batch decryption scheme becomes  $O_\lambda(1)$ . However, the ciphertext now contains  $k$  vectors in  $\mathbb{Z}_{p'}^m$ :

$$|\text{ct}| = km \log(p') = O(k^3 n \log(n)(\log(k) + \log(n)))$$



$$= O(k^3 \lambda (\log^2(\lambda) + \log(k) \log(\lambda))) = O_\lambda(k^3).$$

Finally, the pre-decryption key is simply the sum of the  $k$  PRF keys:

$$|\text{sbk}| = m \log q = O(k^2 n \log^2(n)) = O(k^2 \lambda \log^2(\lambda)) = O_\lambda(k^2).$$

### A.3 Vector Commitment Scheme by Waters et al. [46]

We describe Construction 6.2 by Waters et al. [46]. Let

$$\Pi_{\text{samp}} = (\text{Gen}, \text{GenTD}, \text{Expand}, \text{ExpandLocal})$$

be an  $(n, t, q, s)$ -shifted multi-preimage trapdoor sampler that supports local expansion. Given the vector dimension  $\ell$ , let  $B = B(\lambda, \ell)$  be an upper bound parameter. The vector commitment scheme (SETUP, COMMIT, OPEN, VER) below uses predetermined, distinct  $u_i \in \{0, 1\}^{\lceil \log(\ell) \rceil}$  within the trapdoor sampler below, thus, making  $u_i$  implicit in the input to **Expand** and **ExpandLocal**.

- **VC.SETUP**( $1^\lambda, 1^\ell$ )  $\rightarrow$  **pp**: The setup algorithm samples  $\text{crs}_{\text{samp}} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$  and outputs the public parameters  $\text{pp} = (1^\lambda, \ell, \text{crs}_{\text{samp}})$ .
- **VC.COMMIT**(**pp**,  $x$ )  $\rightarrow$  ( $c, \text{st}$ ): Given the public parameters and a vector  $x \in \mathbb{Z}_q^\ell$ , the commitment algorithm first generates

$$(A_1, \dots, A_\ell, \text{td}) \leftarrow \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}).$$

It then samples

$$\text{col}(v_1, \dots, v_\ell, c) \leftarrow \text{SampleMultPre}(\text{td}, x_1 e_1, \dots, x_\ell e_\ell),$$

where  $e_1 = [1, 0, \dots, 0]^T \in \mathbb{Z}_q^m$ . It outputs  $c$  along with the opening proofs  $(v_1, \dots, v_\ell)$  as state **st**.

- **VC.OPEN**(**st**,  $i$ )  $\rightarrow v_i$ : Given an index  $i \in [\ell]$ , the opening algorithm outputs the opening proof  $v_i$ .
- **VC.VER**(**crs**,  $\sigma, i, x_i, v_i$ )  $\rightarrow 0/1$ : The verification algorithm computes  $A_i = \text{ExpandLocal}(1^\lambda, \text{crs}_{\text{samp}}, i)$  and accepts the opening proof if  $\|v_i\| \leq B$  and  $c = A_i v_i + x_i e_1$ .

The vector commitment scheme satisfies correctness, computational position-binding and statistical hiding properties [46, Theorems 6.3, 6.4, 6.8] based on the short integer solution assumption.

### A.4 Identity Based Encryption (IBE) Scheme by Gentry et al. [31]

The IBE scheme of Gentry et al. [31] uses a random oracle  $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$  to associate identities with public keys of the dual-Regev cryptosystem.

- **IBE.SETUP**( $1^n$ ): The setup algorithm generates a matrix-trapdoor tuple:  $(A, T_A) \leftarrow \text{TrapGen}(1^n, q, m)$  for  $m > n \log(q)$ . The master public key is set to be  $\text{IBE.mpk} \leftarrow A$ , which acts as the dual-Regev shared matrix, whereas the master secret key is set to be  $\text{IBE.msk} \leftarrow (A, T_A)$ .
- **IBE.KEYGEN**(**IBE.msk**,  $r$ ): The key generation algorithm computes  $u = H(r)$  and returns the corresponding decryption key  $\text{sbk}_r \leftarrow \text{SamplePre}(A, T_A, u, \sigma)$ .

- $\text{IBE.ENC}(\text{IBE.mpk}, r, b)$ : To encrypt a bit  $b \in \{0, 1\}$  for identity  $r$ , the encryption algorithm computes  $u = H(r) \in \mathbb{Z}_q^n$  and generates the dual-Regev ciphertext:  $\text{ct} = (\text{ct}[1], \text{ct}[2])$ :

$$\begin{aligned}\text{ct}[2] &\triangleq u^\top s + \epsilon_2 + b \lfloor q/2 \rfloor \in \mathbb{Z}_q^m \\ \text{ct}[1] &\triangleq A^\top s + \epsilon_1 \in \mathbb{Z}_q,\end{aligned}$$

where  $s \leftarrow \mathbb{Z}_q^n$ ,  $\epsilon_1 \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma}$ , and  $\epsilon_2 \leftarrow D_{\mathbb{Z}_q, \sigma}^m$ .

- $\text{IBE.DEC}(\text{sbk}_r, \text{ct})$ : The decryption algorithm outputs

$$\text{Round}(\text{ct}[1] - \text{sbk}_r^\top \text{ct}[2]),$$

where  $\text{Round}(x)$  outputs 0 if  $x \bmod q$  is closer to 0 than to  $\lfloor q/2 \rfloor \bmod q$ .

The scheme satisfies correctness and adaptive security [31, Theorem 7.2].

**Proof Sketch for Lemma 8.** We note that Lemma 8 follows from the proof of [31, Corollary 5.4], along with [31, Lemma 3.1] and [35, Theorem 4.1]. Here, [35, Theorem 4.1] states that  $A^\perp(\mathbf{G})$  has a known basis  $\mathbf{S} \in \mathbb{Z}_q^{m' \times m'}$  such that  $\|\mathbf{S}\|_{\text{GS}} \leq \sqrt{5}$ . On the other hand, by [31, Lemma 3.1], the smoothing parameter  $\eta_\epsilon$  for  $A^\perp(\mathbf{G})$  is bounded by  $\sqrt{5} \cdot \omega(\sqrt{\log(m')})$ . This implies that for  $\sigma \geq \omega(\sqrt{\log(m')})$ , it holds that  $\sigma \geq \eta_\epsilon(A^\perp(\mathbf{G}))$ . Finally, as the columns of  $\mathbf{G}$  generate  $\mathbb{Z}_q^n$ , by the proof of [31, Corollary 5.4], the distribution of  $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$ , where  $\mathbf{v} \leftarrow \mathbb{Z}_q^n$ , is statistically close to  $\mathcal{D}_{\mathbb{Z}_q, \sigma}^{m'}$ , where  $m' = n(\lfloor \log(q) \rfloor + 1)$ .

## Learning with Errors.

**Definition 11 (Learning with Errors – LWE).** *Given a prime  $q$ , a positive integer  $n$ , and a distribution  $\chi$  over  $\mathbb{Z}_q$ , a  $(\mathbb{Z}_q, n, \chi)$ -LWE problem instance is associated with a challenge oracle  $\mathcal{O}$  that is either a noisy pseudo-random sampler  $\mathcal{O}_s$  for a random secret key  $\mathbf{s} \in \mathbb{Z}_q^n$ , or a truly random sampler  $\mathcal{O}_\S$ , behaving as follows:*

- $\mathcal{O}_s$ : outputs samples of the form  $(\mathbf{u}_i, \mathbf{v}_i) = (\mathbf{u}_i, \mathbf{u}_i^\top \mathbf{s} + \mathbf{x}_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ , where  $\mathbf{u}_i \leftarrow \mathbb{Z}_q^n$ ,  $\mathbf{s} \leftarrow \mathbb{Z}_q^n$  is a uniformly-random invariant, and  $\mathbf{x}_i \in \mathbb{Z}_q$  is a fresh sample from  $\chi$ .
- $\mathcal{O}_\S$ : outputs truly uniform random samples from  $\mathbb{Z}_q^n \times \mathbb{Z}_q$ .

An algorithm  $\mathcal{A}$  decides the  $(\mathbb{Z}_q, n, \chi)$ -LWE problem if for some  $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ , the advantage of  $\mathcal{A}$ , defined as

$$\text{Adv}_{\text{LWE}}[\mathcal{A}] = |\Pr[\mathcal{A}^{\mathcal{O}_s} = 1] - \Pr[\mathcal{A}^{\mathcal{O}_\S} = 1]|,$$

is non-negligible in  $\lambda$ .

The  $(\mathbb{Z}_q, n, \chi)$ -LWE assumption states that no PPT algorithm decides the  $(\mathbb{Z}_q, n, \chi)$ -LWE problem.

For some  $\alpha \in (0, 1)$  and a prime  $q$ , we set the noise distribution  $\chi$  to be  $\Psi_\alpha$ , the distribution over  $\mathbb{Z}_q$  of the random variable  $\lfloor qX \rfloor \bmod q$ , where  $X$  is a normal random variable with mean 0 and standard deviation  $\alpha/\sqrt{2\pi}$ . Then, as in Agarwal et al. [2], we assume the hardness of approximating the SIVP and GAPSVP problems in lattices of dimension  $n$  to within approximation factors polynomial in  $n$ . Given these assumptions and [2, Theorem 11], which in turn follows from Regev’s reduction [38], we conclude that deciding the  $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE problem is hard, when  $n/\alpha$  is a polynomial in  $n$  and  $q > 2\sqrt{n}/\alpha$ .

## B Batch Decryption from Attribute Based Encryption

In this section, we present a batch decryption protocol that uses as a black-box a KP-ABE construction with attribute space  $\mathcal{I} = \{0, 1\}^k$  and message space  $\mathcal{M}$ , instantiated with the following class of functions:

$$\mathcal{F}_{k,\ell} = \{f_S: \{0, 1\}^k \rightarrow \{0, 1\} \mid S \subseteq (\{0, 1\}^k)^\ell, |S| = \ell, f_S(x) = 0 \text{ if and only if } x \in S\}.$$

### B.1 Definitions

**Definition 12.** A key-policy attribute-based encryption (KP-ABE) scheme for some class of functions  $\mathcal{F}$  and message space  $\mathcal{M}$  consists of four algorithms:

- $\text{ABE.SETUP}(1^\lambda, \mathcal{F}) \rightarrow (\text{mpk}, \text{msk})$ : The setup algorithm takes as input the security parameter  $\lambda$  and the description of the class of functions  $\mathcal{F}$ , and outputs the master public key  $\text{mpk}$  and the master secret key  $\text{msk}$ .
- $\text{ABE.ENC}(\text{mpk}, x, m) \rightarrow \text{ct}$ : The encryption algorithm takes as input  $\text{mpk}$ , an attribute  $x \in \{0, 1\}^*$  and a message  $m \in \mathcal{M}$ , and outputs an attribute-ciphertext tuple  $(x, \text{ct})$ .
- $\text{ABE.KEYGEN}(\text{msk}, f) \rightarrow \text{sk}_f$ : The key generation algorithm takes as input  $\text{msk}$  and a function  $f \in \mathcal{F}$ , and outputs a secret key  $\text{sk}_f$ .
- $\text{ABE.DEC}(\text{sk}_f, (x, \text{ct})) \rightarrow m$ : The decryption algorithm takes as input a secret key  $\text{sk}_f$  and an attribute-ciphertext tuple  $(x, \text{ct})$ , and if  $f(x) = 0$ , outputs the corresponding message  $m$ .

**Definition 13 (Correctness of KP-ABE, taken from [2]).** For every function  $f \in \mathcal{F}$ , attribute strings  $x \in \{0, 1\}^*$ , where  $f(x) = 0$ , and messages  $m \in \mathcal{M}$ , it holds that

$$\Pr \left[ \text{ABE.DEC}(\text{sk}_f, (x, \text{ct})) = m \mid \begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{ABE.SETUP}(1^\lambda, \mathcal{F}) \\ \text{ct} \leftarrow \text{ABE.ENC}(\text{mpk}, x, m) \\ \text{sk}_f \leftarrow \text{ABE.KEYGEN}(\text{msk}, f) \end{array} \right] > 1 - \text{negl}(\lambda)$$

**Definition 14 (Selective Security of KP-ABE, taken from [2]).** An ABE scheme for a class of functions  $\mathcal{F}_\lambda = \{f : \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda\}$  is selectively secure if for all PPT adversaries  $\mathcal{A}$ , where  $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ , it holds that

$$|\Pr[\text{Game}_{\mathcal{A},0}^{\text{ABE}}(\lambda) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{ABE}}(\lambda) = 1]| \leq \text{negl}(\lambda),$$

where for each  $b \in \{0, 1\}$  and  $\lambda \in \mathbb{N}$ , the game  $\text{Game}_{\mathcal{A},b}^{\text{ABE}}(\lambda)$  is defined as follows:

1.  $(x^* \in \mathcal{X}_\lambda, \text{state}_1) \leftarrow \mathcal{A}_1(\lambda)$ , //  $\mathcal{A}$  commits to the challenge attribute  $x^*$
2.  $(\text{mpk}, \text{msk}) \leftarrow \text{ABE.SETUP}(\lambda)$
3.  $(m_0, m_1, \text{state}_2) \leftarrow \mathcal{A}_2^{\text{KG}(\text{msk}, x^*, \cdot)}(\text{mpk}, \text{state}_1)$  //  $\mathcal{A}$  outputs messages
4.  $\text{ct}^* \leftarrow \text{ABE.ENC}(\text{mpk}, x^*, m_b)$
5.  $b' \leftarrow \mathcal{A}_3^{\text{KG}(\text{msk}, x^*, \cdot)}(\text{ct}^*, \text{state}_2)$  //  $\mathcal{A}$  outputs a guess  $b'$  for  $b$
6. Output  $b' \in \{0, 1\}$ ,

where  $\text{KG}(\text{msk}, x^*, f)$  returns a secret key  $\text{sk}_f = \text{ABE.KEYGEN}(\text{msk}, f)$  if  $f(x^*) \neq 0$  and  $\perp$  otherwise.

A fully secure ABE scheme has a similar definition, except that the adversary determines the challenge attribute  $x^*$  after receiving  $\text{mpk}$  and making a polynomial number of secret key queries. Any selectively secure ABE scheme can be converted into a fully secure one at the cost of an exponential loss in the parameter size [9].

## B.2 The Construction

**B.2.1 BD.Setup( $1^\lambda$ ).** The setup algorithm invokes

$$\text{ABE.SETUP}(1^\lambda, \mathcal{F}_{k,\ell}) \rightarrow (\text{ABE.mpk}, \text{ABE.msk})$$

and outputs  $\text{BD.pk} \leftarrow \text{ABE.mpk}$  and  $\text{BD.sk} \leftarrow \text{ABE.msk}$ .

**B.2.2 BD.Enc( $\text{BD.pk}, r \in \{0,1\}^k, m \in \mathcal{M}$ ).** Parsing  $\text{ABE.mpk} \leftarrow \text{BD.pk}$ , the encryption algorithm calls

$$\text{ABE.ENC}(\text{ABE.mpk}, r, m) \rightarrow \text{ABE.ct}.$$

and outputs  $\text{ct} \leftarrow (r, \text{ABE.ct})$ .

**B.2.3 BD.PreDec( $\text{BD.sk}, \{r_i\}_{i \in [\ell]}$ ).** Parsing  $\text{ABE.msk} \leftarrow \text{BD.sk}$ , the pre-decryption algorithm invokes

$$\text{ABE.KEYGEN}(\text{ABE.msk}, f) \rightarrow \text{ABE.sk}_f,$$

with the function  $f: \{0,1\}^k \rightarrow \{0,1\}$  such that  $f(x) = 0$  if and only if  $x \in \{r_i\}_{i \in [\ell]}$ . It outputs  $\text{BD.sbk} \leftarrow \text{ABE.sk}_f$ .

**B.2.4 BD.Dec( $\text{BD.pk}, \text{BD.sbk}, \{\text{ct}_i\}_{i \in [\ell]}$ ).** The decryption algorithm parses  $\text{BD.pk}$ ,  $\text{BD.sbk}$  and  $(r_i, \text{ABE.ct}_i) \leftarrow \text{ct}_i$ . Then, for each  $i \in [\ell]$ , it outputs

$$\text{ABE.DEC}(\text{sk}_f, (r_i, \text{ABE.ct}_i)) \rightarrow m_i.$$

**B.2.5 Parameter Sizes.** If the decryption key  $\text{sk}_f$  and the ciphertext size of the KP-ABE scheme is succinct in the size of the circuit encoding the function  $f_S$ , *i.e.*, succinct in the size  $\ell$  of the set  $S = \{r_i\}_{i \in [\ell]}$ , we obtain pre-decryption keys and ciphertexts that are also succinct in  $\ell$ . For instance, given the KP-ABE scheme of Boneh et al. [14], the predecryption key becomes a  $2m \times m$  low-norm matrix, where  $m = O(n \log(q))$  and  $\log(q)$  grows linearly with the circuit depth  $d$ , implying an overall predecryption key size of...

**B.2.6 Correctness.** For  $\text{BD.pk} \leftarrow \text{ABE.mpk}$ , a correct ciphertext  $\text{ct}_i \leftarrow (r_i \in \{0,1\}^k, \text{ABE.ct}_i)$ , where  $\text{ABE.ct}_i = \text{ABE.ENC}(\text{ABE.mpk}, r_i, m_i)$ , and a correct predecryption key  $\text{BD.sbk} \leftarrow \text{sk}_f$ , where  $\text{sk}_f = \text{ABE.KEYGEN}(\text{ABE.msk}, f)$  for the function  $f \in \mathcal{F}_{k,\ell}$  such that  $f(x) = 0$  if and only if  $x \in \{r_i\}_{i \in [\ell]}$ , by the correctness of the KP-ABE scheme,

$$\begin{aligned} & \text{BD.DEC}(\text{BD.pk}, \text{BD.sbk}, \{\text{ct}_i\}_{i \in [\ell]}) \\ &= \{\text{ABE.DEC}(\text{sk}_f, (r_i, \text{ABE.ct}_i))\}_{i \in [\ell]} = \{m_i\}_{i \in [\ell]} \end{aligned}$$

except with negligible probability.

### B.3 Security

**Theorem 3.** *The BD scheme instantiated with an adaptively secure ABE protocol satisfies security.*

*Proof.* Suppose that a PPT adversary  $\mathcal{A}$  has a non-negligible advantage  $\epsilon$  against the BD game of Definition 3. Then, we construct an adversary  $\mathcal{B}$  with advantage at least  $\epsilon$  in the ABE security game. Here,  $\mathcal{B}$  simulates the view of  $\mathcal{A}$  as follows:

- $\mathcal{B}$  receives  $\text{ABE.mpk}$  for the class of functions  $\mathcal{F}_{k,l}$  from the challenger Ch and sends  $\text{BD.pk} = \text{ABE.mpk}$  to  $\mathcal{A}$ .
- Whenever  $\mathcal{A}$  sends a pre-challenge query for the identities  $(r_1, \dots, r_\ell)$ ,  $\mathcal{B}$  queries the oracle  $\text{KG}(\text{ABE.msk}, \cdot)$  with the function  $f: \{0, 1\}^k \rightarrow \{0, 1\}$  such that  $f(x) = 0$  if and only if  $x \in \{r_i\}_{i \in [\ell]}$ . Upon receiving the secret key  $\text{sk}_f$ , it forwards  $\text{BD.sbk} = \text{sk}_f$  to  $\mathcal{A}$ .
- When  $\mathcal{A}$  sends the two messages  $(m_0, m_1)$  and the identity  $r^*$  it would like to be challenged on,  $\mathcal{B}$  sets its challenge attribute as  $x^* = r^*$  and sends  $m_0, m_1$  and  $x^*$  to Ch. Upon receiving a ciphertext  $c^*$  from Ch, it forwards  $(r^*, c^*)$  to  $\mathcal{A}$ .
- For the post-challenge queries,  $\mathcal{B}$  repeats the same approach as the pre-challenge queries. If  $r^* \in (r_1, \dots, r_\ell)$  for a sequence of post-challenge queries, then  $\mathcal{B}$  aborts and returns a random bit.
- Once  $\mathcal{A}$  outputs a bit  $b'$ ,  $\mathcal{B}$  outputs the same bit.

We next analyze the reduction. Conditioned on  $\mathcal{B}$  not aborting,  $\text{KG}(\text{ABE.msk}, \cdot)$  responds with the correct secret key  $\text{sk}_f = \text{ABE.KEYGEN}(\text{ABE.msk}, f)$ , which is the same as the predecryption key  $\text{BD.sbk}$  corresponding to the sequence of identities defining the function  $f$ . Hence, the view  $\mathcal{B}$  provides to  $\mathcal{A}$  is statistically indistinguishable from the view of a real interaction with the challenger of the BD scheme. Hence,  $\mathcal{A}$  has advantage  $\epsilon$  against the BD game of Definition 3. Now, when  $\mathcal{A}$  wins the BD game in Definition 3,  $\mathcal{B}$  wins the ABE game in Definition 14. Therefore,  $\mathcal{B}$  has advantage at least  $\epsilon$  against the ABE game of Definition 14.  $\square$

### B.4 Parameter Sizes

We next estimate the public key, ciphertext and pre-decryption key sizes achieved by the BD systems instantiated with three LWE-based KP-ABE schemes chosen due to their succinct secret keys: BGGHNSVV14 [14], CW23 [24], and Wee24 [47].

**The KP-ABE scheme by BGGHNSVV14 [14].** The KP-ABE scheme by BGGHNSVV14 supports boolean circuits with inputs in  $\{0, 1\}$ , as well as arithmetic circuits with inputs in  $\mathbb{Z}_q$ . To support circuits of depth  $d$  and  $w$ -way multiplication gates, where all but one of the inputs are in  $[-p, p]$ , the protocol chooses  $n$  such that  $n \geq d^{1/\epsilon}(2 \log_2(n) + w \log(p))^{1/\epsilon}$ , sets  $q = 2^{n^\epsilon}$  and  $m = O(n \log q)$ , where  $\epsilon$  is the analogue of the parameter  $\epsilon$  in Appendix A.2.

We can express the functions  $f_S(x) = \prod_{i \in [\ell]} (x - r_i)$ , where  $x, r_i \in \{0, 1\}^k$ , as boolean circuits with  $k$ -bit inputs as follows. Let  $x^j, j \in [k]$ , denote the bits of the input identity  $x$ , and  $r_i^j, j \in [k]$ , denote the bits of the identity  $r_i$ . Let  $\neg, \vee$  and  $\wedge$  denote the ‘complement’, ‘or’, and ‘and’ operations respectively. Then, the circuit computes

$$\neg \left( \bigvee_{i=1}^{\ell} \left( \bigwedge_{j=1}^k (x^j \oplus r_i^j) \right) \right)$$

Note that this circuit can be expressed as depth  $O(\log(k + \ell)) = O(\log(k))$  boolean and arithmetic circuits with  $k$ -bit inputs, where the fan-in is 2, *i.e.*,  $w = 2$ , and each input is a bit, *i.e.*,  $p = 2$  (recall that  $k > \ell$  for the KP-ABE schemes). Finally, this implies  $n = O(\log^{1/\epsilon}(k))$ .

Now, the KP-ABE secret keys are low-norm matrices in  $\mathbb{Z}^{2m \times m}$ , sampled from a Gaussian distribution with width  $\sigma = \omega(\sqrt{n} \log(m))$ . Then, given  $\log(q) = n^\epsilon = O(\log(k))$ , the public keys scale as

$$|\text{pp}| = O(knm \log(q)) = O(kn^2 \log^2(q)) = O(k\lambda^2 \log^2(k)),$$

the pre-decryption keys scale as

$$|\text{sbk}| = O(m^2 \log(\sigma)) = O(n^2 \log^2(q) \log(n)) = O(\lambda^2 \log^2(k)),$$

and ciphertexts scale as

$$|\text{ct}| = O(km \log(q)) = O(kn \log^2(q)) = O(k\lambda \log^2(k)).$$

We ignore the  $\epsilon$  terms in the power; otherwise, the power of  $\log(k)$  would be  $2 + 2/\epsilon$ .

**The KP-ABE scheme by CW23.** The KP-ABE scheme by CW23 supports boolean circuits with inputs in  $\{0, 1\}^k$  and achieve keys of size  $O(\text{poly}(\lambda))$ , independent of the circuit depth. More specifically, CW23 achieves parameters of size

$$\begin{aligned} |\text{mpk}| &= O(kn_1^2 \log^2(q)) \\ |\text{sk}| &= O(n_0 \lambda \log^2(p)) \\ |\text{ct}| &= O(kn_1 \log^2(q)) \end{aligned}$$

where  $n_0 = O(\lambda)$ ,  $n_1 = O(\lambda d^{1/\epsilon})$  and  $\log(q) = O(d\lambda)$  are analogous to the parameters  $n$  and  $\log(q)$  in BGGHNSVV14,  $d$  is the circuit depth, and  $\lambda$  is the security parameter. Note that  $\log(p) = O(\lambda)$  has no dependence on the circuit depth.

Now, as in BGGHNSVV14, we can express the functions  $f_S(x) = \prod_{i \in [q]} (x - r_i)$ , where  $x, r_i \in \{0, 1\}^k$ , as depth  $O(\log(k))$  boolean circuits with  $k$ -bit inputs, where the fan-in is 2, *i.e.*,  $w = 2$ , and each input is a bit, *i.e.*,  $p = 2$ . Therefore, the batch decryption scheme based on CW23 achieves parameters of size

$$\begin{aligned} |\text{pk}| &= O(k\lambda^4 \log^2(k)) \\ |\text{sbk}| &= O(\lambda^4) \\ |\text{ct}| &= O(k\lambda^3 \log^2(k)) \end{aligned}$$

We again ignore the  $2/\epsilon$  term in the power.

**The KP-ABE scheme by Wee24.** Using  $\ell$ -succinct LWE, Wee24 can achieve any  $|\text{pk}|$  and  $|\text{ct}|$  size of the form  $O(k^{2\alpha} \text{poly}(\lambda, d))$  and  $k + O(k^{1-\alpha} \text{poly}(\lambda, d))$ , where  $\alpha \in [1/3, 1]$ .

As  $k$  would be  $\beta \log(\ell)$ , the public key and ciphertext sizes are comparable to those of our ‘vanilla’ batch decryption scheme without the solution in Section 4.4 (besides having larger log terms as  $k > \ell$ ). However, the BGGHNSVV14 construction has quadratically larger pre-decryption keys, and the solution in Section 4.4 does not help the KP-ABE based constructions remove the linear dependency on  $k$ . Therefore, the KP-ABE based constructions have larger public keys and ciphertexts compared to our final batch decryption protocol.

## C Batch Decryption from Trilinear Maps

In this section, we present a batch decryption scheme based on trilinear maps.

### C.1 Background on Trilinear Maps and Generic Group Model

Let the product notation  $(\cdot) : \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_3 \rightarrow \mathbb{G}_t$  denote a trilinear map, where  $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$  and  $\mathbb{G}_t$  are multiplicative groups of prime order  $p$  with generators  $G_1, G_2, G_3$ , and  $G_T = G_1 \cdot G_2 \cdot G_3$ , respectively. Let  $H : \mathbb{G}_t \rightarrow \{0, 1\}^{|\mathcal{M}|}$  denote a one-way function. The generic group model considers adversaries that do not rely on the concrete representations of the group elements but instead operate using only generic group operations. This model is designed to capture algebraic attacks executed by such adversaries.

The following description of Shoup's GGM [43] is adapted from [49,1]. Let  $p \in \mathbb{Z}$  be a positive integer, and let  $S \subseteq \{0, 1\}^*$  be a set of strings with cardinality at least  $p$ , with a known upper bound. A group  $\mathbb{G}$  of prime order  $p$  is generically represented by sampling a random injection  $L : \mathbb{Z}_p \rightarrow S$ , known as the labeling function. This function encodes the string representation of  $xG_1$ , where  $G_1 \in \mathbb{G}$  is a fixed generator of  $\mathbb{G}$ . The model provides the following query functionalities:

- **Labeling queries:** Given  $x \in \mathbb{Z}_p$ , returns  $L(x)$ .
- **Group operations:** Given  $(\ell_1, \ell_2, a_1, a_2) \in S^2 \times \mathbb{Z}_p^2$ , if there are  $x_1, x_2 \in \mathbb{Z}_p$  such that  $L(x_1) = \ell_1$  and  $L(x_2) = \ell_2$ , returns  $L(a_1x_1 + a_2x_2)$ . Otherwise, returns  $\perp$ .

The labeling function  $L$  need not be precomputed in its entirety; instead, it can be generated dynamically as new queries arrive. As such, it can be modeled as a dictionary indexed by  $x \in \mathbb{Z}_p$ .

In a trilinear group setting, the model also supports an additional query type:

- **Mapping query:** Given  $(\ell_1, \ell_2, \ell_3) \in S_1 \times S_2 \times S_3$ , if there are  $x_1, x_2, x_3 \in \mathbb{Z}_p$  such that  $L_1(x_1) = \ell_1$ ,  $L_2(x_2) = \ell_2$ , and  $L_3(x_3) = \ell_3$ , returns  $L_T(x_1 \cdot x_2 \cdot x_3)$ . Otherwise, returns  $\perp$ .

### C.2 The Construction

**C.2.1 BD.Setup( $1^\lambda$ ).** The setup algorithm samples  $\alpha \leftarrow \mathbb{Z}_p$  and outputs the secret key  $\text{BD.sk} \leftarrow \alpha$ . It calculates  $A_1 = \alpha G_1$ , and  $B_i := \alpha^i G_2$  for  $i \in [\ell - 1]$ , and outputs the public key  $\text{BD.pk} \leftarrow (A_1, B_1, \dots, B_{\ell-1})$ .

**C.2.2 BD.Enc( $\text{BD.pk}, r \in \mathbb{Z}_p, m \in \{0, 1\}^k$ ).** Parsing  $(A_1, B_1, \dots, B_{\ell-1}) \leftarrow \text{BD.pk}$ , the encryption algorithm samples  $t \leftarrow \mathbb{Z}_p$  and outputs  $\text{ct} = (r, \text{ct}[1], \text{ct}[2])$ :

$$\text{ct}[1] = tA_1 + rtG_1 \quad \text{ct}[2] = m \oplus H(tG_T)$$

**C.2.3 BD.PreDec( $\text{BD.sk}, \{r_i\}_{i \in [\ell]}$ ).** Parsing  $\alpha \leftarrow \text{BD.sk}$ , the predecryption algorithm outputs

$$\text{BD.sbk} \leftarrow \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3.$$

**C.2.4 BD.Dec(BD.pk, BD.sbk, {ct<sub>i</sub>}<sub>i∈[ℓ]</sub>).** After parsing the public key and the ciphertexts ct<sub>i</sub> = (r<sub>i</sub>, ct<sub>i</sub>[1], ct<sub>i</sub>[2]), for each i ∈ [ℓ], the decryption algorithm finds the coefficients {f<sub>i</sub><sup>j</sup>}<sub>j=0,...,ℓ-2</sub>, of the polynomials

$$f_i(X) = X^{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j X^j = \prod_{j \in [\ell], j \neq i} (X + r_j).$$

Then, for each i ∈ [ℓ], it calculates

$$f_i(\alpha)G_2 = \prod_{j \in [\ell], j \neq i} (\alpha + r_j)G_2 = B_{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j B_j,$$

where B<sub>0</sub> = G<sub>2</sub>. Finally, for i ∈ [ℓ], it outputs

$$m'_i \leftarrow \text{ct}_i[2] \oplus H(\text{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BD.sbk}).$$

**C.2.5 Correctness.** For a correct ciphertext ct<sub>i</sub> = (r<sub>i</sub>, ct<sub>i</sub>[1], ct<sub>i</sub>[2]), where ct<sub>i</sub>[1] = tA<sub>1</sub> + r<sub>i</sub>tG<sub>1</sub> and ct<sub>i</sub>[2] = m<sub>i</sub> ⊕ H(tG<sub>T</sub>), and correct predecryption key sbk =  $\frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3$ , it holds that

$$\begin{aligned} m'_i &= \text{ct}_i[2] \oplus H(\text{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BD.sbk}) \\ &= m_i \oplus H(tG_T) \oplus H\left((\alpha + r_i)tG_1 \cdot \prod_{j \in [\ell], j \neq i} (\alpha + r_j)G_2 \cdot \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3\right) \\ &= m_i \oplus H(tG_T) \oplus H(tG_T) = m_i \end{aligned}$$

### C.3 Thresholdizing the Secret Key

To thresholdize the pre-decryption key  $\frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3$ , where r<sub>1</sub>, ..., r<sub>ℓ</sub> are pair-wise distinct, we can first express it as a sum of partial fractions:

$$\frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3 = \sum_{i \in [\ell]} \frac{\beta_i}{\alpha + r_i} G_3$$

for some β<sub>1</sub>, ..., β<sub>ℓ</sub> ∈ ℤ<sub>p</sub>. Now, suppose α is t-out-of-n secret shared among n parties. Then for i ∈ [ℓ], it suffices to describe a secure protocol for computing (1/(α + r<sub>i</sub>))G<sub>3</sub> among a quorum of t parties. This problem is the same as computing a threshold signature in the Boneh-Boyen signature scheme [8], and Wang et al. [45] give an efficient two-round protocol for this problem. Briefly, the parties first jointly compute and publish β = s(α + r<sub>i</sub>) ∈ ℤ<sub>p</sub> in the clear, where s ← ℤ<sub>p</sub> is additively secret shared among them, say s = s<sub>1</sub> + ... + s<sub>t</sub>. Each party then publishes P<sub>i</sub> = (s<sub>i</sub>/β)G<sub>3</sub>. Then,  $\sum_{i \in [t]} P_i = (1/(\alpha + r_i))G_3$ , as required.

### C.4 Security

**Theorem 4.** For all ℓ ∈ ℕ and unbounded adversaries  $\mathcal{A}$  making at most q queries to the group oracle, hash oracle, and for key computation, it holds that

$$\Pr[\text{Game}_{\mathcal{A},0}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1]$$



$$\leq \binom{q + \ell + 3}{2} \frac{\ell}{p} = \text{negl}(\lambda),$$

for  $p = O(2^\lambda)$ ,  $\ell, q = \text{poly}(\lambda)$ .

For the proof of Theorem 4, we follow the GGM proof blueprint and notation in [1], applied to our protocol. In a GGM proof, the challenger becomes responsible for simulating the GGM oracle by selecting secret values  $x_1, \dots, x_n \in \mathbb{Z}_p$  and revealing  $L(f(x_1, \dots, x_n))$  to the adversary for some function  $f$ . To show security, we also use the symbolic model from [43], where instead of sampling each secret  $x_i \in \mathbb{Z}_p$  explicitly, Ch replaces it with an indeterminate  $X_i$ . Then, rather than returning  $L(f(x_1, \dots, x_n))$  to the adversary, the challenger instead leaks  $L(f(X_1, \dots, X_n))$ . Before we present the proof, we describe the security game in GGM and the symbolic model respectively.

**Security Game in the Generic Group Model:**  $\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$

**Setup:** Let  $S_1 = S_2 = S_3 = S_T = \{0, 1\}^{p'}$  where  $2^{p'} \geq p$ , and  $p$  is the group order. The challenger first initializes the maps including the labeling functions  $L_i : \mathbb{Z}_p \rightarrow S_i$ ,  $i \in \{1, 2, 3, T\}$ , and the hash function  $H : S_T \rightarrow \{0, 1\}^k$ . It then runs  $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{BD.pk}, \text{BD.sk})$ , having taken the security parameter  $\lambda$  and the batch size  $\ell$  as input. During the call to  $\text{BD.SETUP}$ , it performs the following steps:

- Sample  $\alpha \leftarrow \mathbb{Z}_p$ .
- Sample  $e_\alpha^1 \leftarrow S_1$ , and set  $L_1(\alpha) \triangleq e_\alpha^1$  and  $S_1 \triangleq S_1 \setminus \{e_\alpha^1\}$ .
- For  $i \in [\ell - 1]$ , sample  $e_{\alpha^i}^2 \leftarrow S_2$ , and set  $L_2(\alpha^i) \triangleq e_{\alpha^i}^2$  and  $S_2 \triangleq S_2 \setminus e_{\alpha^i}^2$ .

It sets  $\text{BD.pk} = (L_1(\alpha), L_2(\alpha), \dots, L_2(\alpha^{\ell-1}))$  and sends  $\text{BD.pk}$  to  $\mathcal{A}$ .

**Group and hash oracle queries:**

- **Labeling query:** For a group  $G_i$ , upon receiving  $x \in \mathbb{Z}_p$ , if  $x \notin L_i$ , Ch samples  $e \leftarrow S_i$ , sets  $L_i(x) \triangleq e$  and updates  $S_i \triangleq S_i \setminus \{e\}$ . It then sends  $L_i(x)$  to  $\mathcal{A}$ .
- **Group operation:** Upon receiving  $(e_1, e_2, a_1, a_2) \in (\{0, 1\}^{p'})^2 \times \mathbb{Z}_p^2$ , if there are  $x_1, x_2 \in \mathbb{Z}_p$  such that  $L_i(x_1) = e_1$  and  $L_i(x_2) = e_2$ , Ch computes  $x_3 = a_1 x_1 + a_2 x_2$ , does a labeling query for  $x_3$ , and sends  $L_i(x_3)$  to  $\mathcal{A}$ .
- **Map operation:** Upon receiving  $(e_1, e_2, e_3)$ , if there are  $x_1, x_2, x_3 \in \mathbb{Z}_p$  such that  $L_1(x_1) = e_1$ ,  $L_2(x_2) = e_2$  and  $L_3(x_3) = e_3$ , Ch computes  $x_T = x_1 \cdot x_2 \cdot x_3$ , does a labeling query for  $x_T$ , and sends  $L_T(x_T)$  to  $\mathcal{A}$ .
- **Hash query:** Upon receiving  $e$ , if there is  $x \in \mathbb{Z}_p$  such that  $L_T(x) = e$ , and  $e \notin H$ , Ch samples  $h \leftarrow \mathbb{Z}_{p'}$ , sets  $H(e) \triangleq h$ , and sends  $h$  to  $\mathcal{A}$ .

**Pre and post-challenge queries:** Upon receiving a list of  $\ell$  identities  $(r_1, \dots, r_\ell) \subseteq \mathcal{I}^\ell$  from  $\mathcal{A}$ , Ch does the following:

- If this is for a post-challenge query and the challenge identity  $r^* \in (r_1, \dots, r_\ell)$ , it halts the game.
- If  $x \triangleq \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} \notin L_3$ , it does a labeling query for  $x$ .
- It sends the predecryption key  $\text{BD.sbk} \triangleq L_3(x)$  to  $\mathcal{A}$ .

**Challenge round:** Upon receiving the messages  $m_0, m_1 \in \mathbb{Z}_{p'}$  and the challenge identity  $r^*$  from  $\mathcal{A}$ , if a pre-challenge query has already been made with a sequence of identities including  $r^*$ , Ch halts the game. Otherwise, it does the following:

- It samples  $t^* \leftarrow \mathbb{Z}_q$  and does a group operation  $(A_1, G_1, t^*, r^*t^*)$  for  $L_1$ . It sets the output to be  $\text{ct}^*[1]$ .
- It does a group operation  $(1, G_T, 1, t^*)$  for  $L_T$  with output  $e$ . It then does a hash query for  $e$  with output  $\text{mask}$ . It sets  $\text{ct}^*[2]$  to be  $m_b \oplus \text{mask}$ .
- It sends the ciphertext  $\text{ct}^* = (r^*, \text{ct}^*[1], \text{ct}^*[2])$  to  $\mathcal{A}$ .

**Output:** Once  $\mathcal{A}$  halts and outputs a bit  $b'$ , the game ends and outputs  $b'$ .

**Security Game in the Symbolic Model:**  $\text{Game}_{\mathcal{A},b}^{\text{BD},\text{SM}}(1^\lambda, 1^\ell)$

**Setup:** Let  $S_1 = S_2 = S_3 = S_T = \{0, 1\}^{p'}$  where  $2^{p'} \geq p$ , and  $p$  is the group order. The challenger first initializes the maps including the labeling functions  $L_i : \mathbb{Z}_p \rightarrow S_i$ ,  $i \in \{1, 2, 3, T\}$ , and the hash function  $H : S_T \rightarrow \{0, 1\}^k$ . It then runs  $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{BD.pk}, \text{BD.sk})$ , having taken the security parameter  $\lambda$  and the batch size  $\ell$  as input. During the call to  $\text{BD.SETUP}$ , it performs the following steps:

- Sample  $e_\alpha^1 \leftarrow S_1$ , and set  $L_1(X_\alpha) \triangleq e_\alpha^1$  and  $S_1 \triangleq S_1 \setminus \{e_\alpha^1\}$ , where  $X_\alpha$  is an indeterminate corresponding to  $\text{BD.sk}$ .
- For  $i \in [\ell - 1]$ , sample  $e_{\alpha^i}^2 \leftarrow S_2$ , and set  $L_2(X_\alpha^i) \triangleq e_{\alpha^i}^2$  and  $S_2 \triangleq S_2 \setminus e_{\alpha^i}^2$ .

It sets  $\text{BD.pk} = (L_1(X_\alpha), L_2(X_\alpha), \dots, L_2(X_\alpha^{\ell-1}))$  and sends  $\text{BD.pk}$  to  $\mathcal{A}$ .

**Group and hash oracle queries:**

- **Labeling query:** For a group  $G_i$ , upon receiving  $x \in \mathbb{Z}_p$ , if  $x \notin L_i$ , Ch samples  $e \leftarrow S_i$ , sets  $L_i(x) \triangleq e$  and updates  $S_i \triangleq S_i \setminus \{e\}$ . It then sends  $L_i(x)$  to  $\mathcal{A}$ .
- **Group operation:** Upon receiving  $(e_1, e_2, a_1, a_2) \in (\{0, 1\}^{p'})^2 \times \mathbb{Z}_p^2$ , if there are polynomials  $x_1, x_2 \in \mathbb{Z}_p[*]$  such that  $L_i(x_1) = e_1$  and  $L_i(x_2) = e_2$ , Ch computes the polynomial  $x_3 = a_1x_1 + a_2x_2$ , does a labeling query for  $x_3$ , and sends  $L_i(x_3)$  to  $\mathcal{A}$ . Otherwise, it sends  $\perp$  to  $\mathcal{A}$ .
- **Map operation:** Upon receiving  $(e_1, e_2, e_3)$ , if there are polynomials  $x_1, x_2, x_3 \in \mathbb{Z}_p[*]$  such that  $L_1(x_1) = e_1$ ,  $L_2(x_2) = e_2$  and  $L_3(x_3) = e_3$ , Ch computes the polynomial  $x_T = x_1 \cdot x_2 \cdot x_3$ , does a labeling query for  $x_T$ , and sends  $L_T(x_T)$  to  $\mathcal{A}$ . Otherwise, it sends  $\perp$  to  $\mathcal{A}$ .
- **Hash query:** Upon receiving  $e$ , if there is a polynomial  $x \in \mathbb{Z}_p[*]$  such that  $L_T(x) = e$ , and  $e \notin H$ , Ch samples  $h \leftarrow \mathbb{Z}_{p'}$ , sets  $H(e) \triangleq h$ , and sends  $h$  to  $\mathcal{A}$ .

**Pre and post-challenge queries:** Upon receiving a list of  $\ell$  identities  $(r_1, \dots, r_\ell) \subseteq \mathcal{I}^\ell$  from  $\mathcal{A}$ , Ch does the following:

- If this is for a post-challenge query and the challenge identity  $r^* \in (r_1, \dots, r_\ell)$ , it halts the game.
- Let  $X_{\text{sbk}} \triangleq \frac{1}{\prod_{i \in [\ell]} (X_\alpha + r_i)}$  be a Laurent polynomial with the indeterminate  $X_\alpha$ . If  $X_{\text{sbk}} \notin L_3$ , Ch does a labeling query for  $X_{\text{sbk}}$ .

- It sends the predecryption key  $\text{BD.sbk} \triangleq L_3(X_{\text{sbk}})$  to  $\mathcal{A}$ .

**Challenge round:** Upon receiving the messages  $m_0, m_1 \in \mathbb{Z}_{p'}$  and the challenge identity  $r^*$  from  $\mathcal{A}$ , if a pre-challenge query has already been made with a sequence of identities including  $r^*$ , Ch halts the game. Otherwise, it does the following:

- Let  $X_{t^*}$  be an indeterminate. Ch sets  $L_1(X_{t^*}X_\alpha + r^*X_{t^*}) \leftarrow \{0, 1\}^{p'}$  and  $\text{ct}^*[1] = L_1(X_{t^*}X_\alpha + r^*X_{t^*})$ .
- It then sets  $L_T(X_{t^*}) \leftarrow \{0, 1\}^{p'}$  and does a hash query for  $L_T(X_{t^*})$  with output **mask**. It sets  $\text{ct}^*[2] = m_b \oplus \text{mask}$ .
- It sends the ciphertext  $\text{ct}^* = (r^*, \text{ct}^*[1], \text{ct}^*[2])$  to  $\mathcal{A}$ .

**Output:** Once  $\mathcal{A}$  halts and outputs a bit  $b'$ , the game ends and outputs  $b'$ .

Proof of Theorem 4 relies on the following lemmas:

**Lemma 23.** For all  $\ell \in \mathbb{N}$  and all unbounded adversaries  $\mathcal{A}$ ,

$$\left| \Pr[\text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| = 0.$$

*Proof.* We describe an intermediate hybrid game  $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell)$  such that  $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell) \equiv \text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell)$  for  $b \in \{0, 1\}$ , which implies the lemma. Here,  $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell)$  is the same as  $\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell)$  except that  $\text{ct}^*[2]$  is computed as follows:

- The challenger defines an indeterminate  $X_u$ , sets  $L_T(X_u) \leftarrow S_T$  and updates  $S_T \triangleq S_T \setminus L_T(u)$ .
- It does a hash query for  $L_T(X_u)$  which outputs **mask**. It sets  $\text{ct}^*[2] = m_b \oplus \text{mask}$ .

Suppose the adversary makes  $n$  queries for the predecryption key with the identities  $r_i, i \in [n\ell]$ . In this case, it holds the following Laurent polynomials:

$$\begin{aligned} L_1 &= \{1, X_\alpha, X_{t^*}(X_\alpha + r^*)\} \\ L_2 &= \{X_\alpha^i\}_{i=0,\dots,\ell-1} \\ L_3 &= \left\{ 1, \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)} \right\}_{j=0,\dots,n-1} \end{aligned}$$

The polynomial used to calculate  $\text{ct}^*[2]$  is  $X_{t^*}$ , and in order to prove the equivalence of the adversary's views in  $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}$  and  $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}$ , we argue that  $X_{t^*}$  is linearly independent of the polynomials in  $L_1 \otimes L_2 \otimes L_3$ . Towards contradiction, suppose this is not the case. Since  $X_{t^*}$  appears only as part of the monomials  $X_{t^*}(X_\alpha + r^*)$ , it must be that

$$X_{t^*} = X_{t^*}(X_\alpha + r^*) \sum_{j=0}^n f_j(X_\alpha) \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)},$$

where  $f_j(\cdot)$  are degree  $\leq (\ell - 1)$  polynomials in  $X_\alpha$ . In other words,

$$\frac{1}{(X_\alpha + r^*)} = \sum_{j=0}^n f_j(X_\alpha) \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)},$$

Now, for this equality to be satisfied,  $r^*$  must be among the values  $\{r_i\}_{i \in [n\ell]}$ , as otherwise  $f_j(X_\alpha)$  would have to be a rational function. This is a contradiction.  $\square$

**Lemma 24.** For all  $\ell \in \mathbb{N}$  and unbounded  $\mathcal{A}$  making at most  $q$  queries to the group oracle, hash oracle, and for key computation, for  $b \in \{0, 1\}$ ;

$$\left| \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| \leq \binom{q + \ell + 3}{2} \frac{\ell}{p}.$$

*Proof.* Note that the challenger can create a perfect simulation of the game  $\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$  by sampling values from  $\mathbb{Z}_p$  uniformly at random at the end of the game  $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell)$  for the involved indeterminates, unless the sampled values cause different polynomials to assume the same value. By the Schwartz-Zippel lemma [50,41], the probability of this event for two polynomials of degree  $(d_1, d_2)$  is at most  $\max(d_1, d_2)/p$ . Now, when the adversary makes  $q$  predecryption queries, there will be  $q$  Laurent polynomials from the predecryption queries,  $\ell + 1$  polynomials from setup (including 1) and 2 polynomials from encryption, implying a total of  $q + \ell + 3$  polynomials across all four groups. Since the largest degree across the polynomials is  $\ell$ , by the union bound, the probability that different polynomials assume the same value can be upper-bounded by

$$\binom{q + \ell + 3}{2} \frac{\ell}{p}.$$

□

*Proof of Theorem 4.* Since

$$\left| \Pr[\text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| = 0$$

by Lemma 23, and

$$\left| \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| \leq \mu \triangleq \binom{q + \ell + 3}{2} \frac{\ell}{p}$$

by Lemma 24, it holds that

$$\text{Game}_{\mathcal{A},0}^{\text{BD,GGM}}(1^\lambda, 1^\ell) \approx_\mu \text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) \equiv \text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) \approx_\mu \text{Game}_{\mathcal{A},1}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$$

□